Table of Contents

PREAMBLE

DO NOT REMOVE THE LINE BELOW

```
clear; close all;
```

==========

==========

1. COMMENTING

==========

==========

MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE

```
type('eel3135_lab06_comment.m')
```

```
%% USER-DEFINED VARIABLES
clear
close all
clc

%% DEFINE FILTER
N = 10;
```

```
h = (1/N)*ones(N,1);

% <-- Answer Question: What is the impulse response of this filter?
%     Use d in place of delta.
% The impulse response of this filter is h[n] = (1/10)(d[n] + d[n-1] + ... +
d[n-9])

% COMPUTE THE DTFT
n = 0:(N-1);
w = -pi:pi/5000:pi;
H = DTFT(h,w);

% PLOT THE IMPULSE RESPONSE AND DTFT
figure
subplot(3,1,1)
stem(n,h)
xlim([-0.5 20.5])
title('Impulse Response of h')
xlabel('Time Index (n)')
ylabel('Amplitude')
subplot(3,1,2)
plot(w,abs(H))
grid on;
title('Magnitude Response of H')
ylabel('Magnitude [rad]')
xlabel('Normalized Angular Frequency [rad/s]')
subplot(3,1,3)
plot(w,angle(H))
grid on;
title('Phase Response of H')
ylabel('Phase [rad]')
xlabel('Normalized Angular Frequency [rad/s]')


function H = DTFT(x,w)
%  ===> Describe function here <===
% Computes the DTFT of signal x
% evaluated at normalized angular frequencies w.
%
%  Inputs:
%    x = discrete-time signal (vector)
%    w = frequency vector (radians/sample)
%
%  Output:
%    H = DTFT of x evaluated at frequencies w

    H = zeros(length(w),1);
    for nn = 1:length(x)
        H = H + x(nn).*exp(-1j*w.'*(nn-1));
    end

end
```

==========

==========

2. DTFT OF COMMON FUNCTIONS

==========

==========

---------- 2(a) ---------- x[n] = δ[n−1]+δ[n−2]+δ[n−3]

```
n = 0:24;

% build x[n]
x = zeros(size(n));
x(n==1) = 1;
x(n==2) = 1;
x(n==3) = 1;

% DTFT
w = -pi:pi/5000:pi;
X = DTFT(x(:), w);   % make x a column to match DTFT loop


% plots
figure
subplot(3,1,1)
stem(n, x, 'filled')
xlim([-0.5 24.5])
title('Time-Domain Signal x[n]')
xlabel('Time Index (n)')
ylabel('Amplitude')

subplot(3,1,2)
plot(w, abs(X))
grid on
title('Magnitude Response |X(e^{j\omega})|')
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')

subplot(3,1,3)
plot(w, angle(X))
grid on
title('Phase Response \angle X(e^{j\omega})')
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Phase (rad)')

%classify
disp('2(a): predominantly LOW frequency (peak at 0 rad)')
```

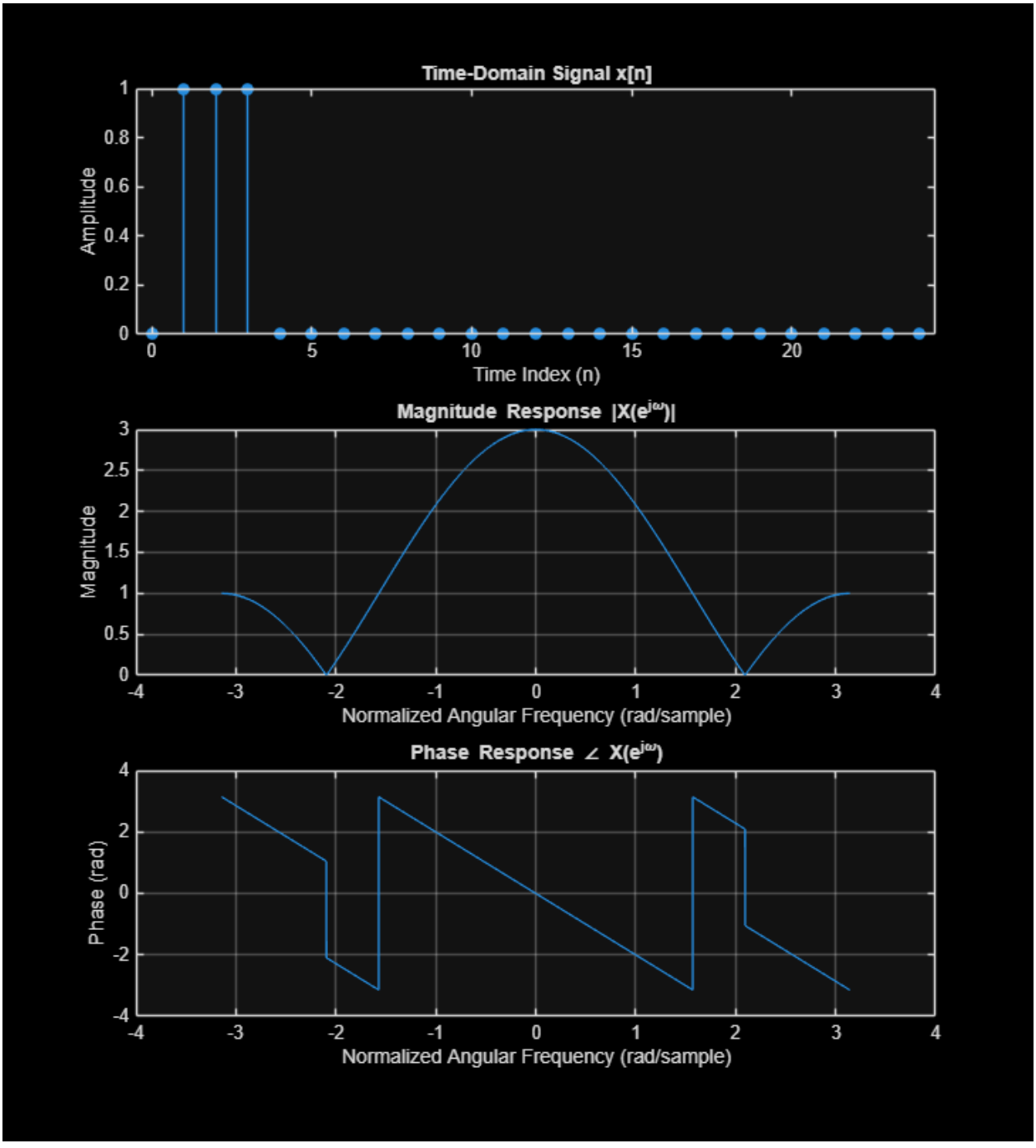
```
% ------------------------------------------------------------------
% 2(b)
% ------------------------------------------------------------------
% (b) x[n] = δ[n-2]+δ[n-3]+δ[n-4]
n = 0:24;                       % required sample range

% Construct x[n] (three shifted unit impulses)
x = zeros(size(n));
x(n==2) = 1;
x(n==3) = 1;
x(n==4) = 1;

% Frequency vector for DTFT
w = -pi:pi/5000:pi;

% Compute DTFT of x[n]
X = DTFT(x(:), w);

% -------------------- Plots --------------------
figure

% Time-domain signal
subplot(3,1,1)
stem(n, x, 'filled')
xlim([-0.5 24.5])
xlabel('Time Index (n)')
ylabel('Amplitude')
title('Time-Domain Signal x[n]')

% Magnitude spectrum
subplot(3,1,2)
plot(w, abs(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude Response |X(e^{j\omega})|')

% Phase spectrum
subplot(3,1,3)
plot(w, angle(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Phase (rad)')
title('Phase Response \angle X(e^{j\omega})')

% Visual classification
disp('2(b): predominantly LOW frequency (peak at 0 rad)')


% ------------------------------------------------------------------
% 2(c)
```

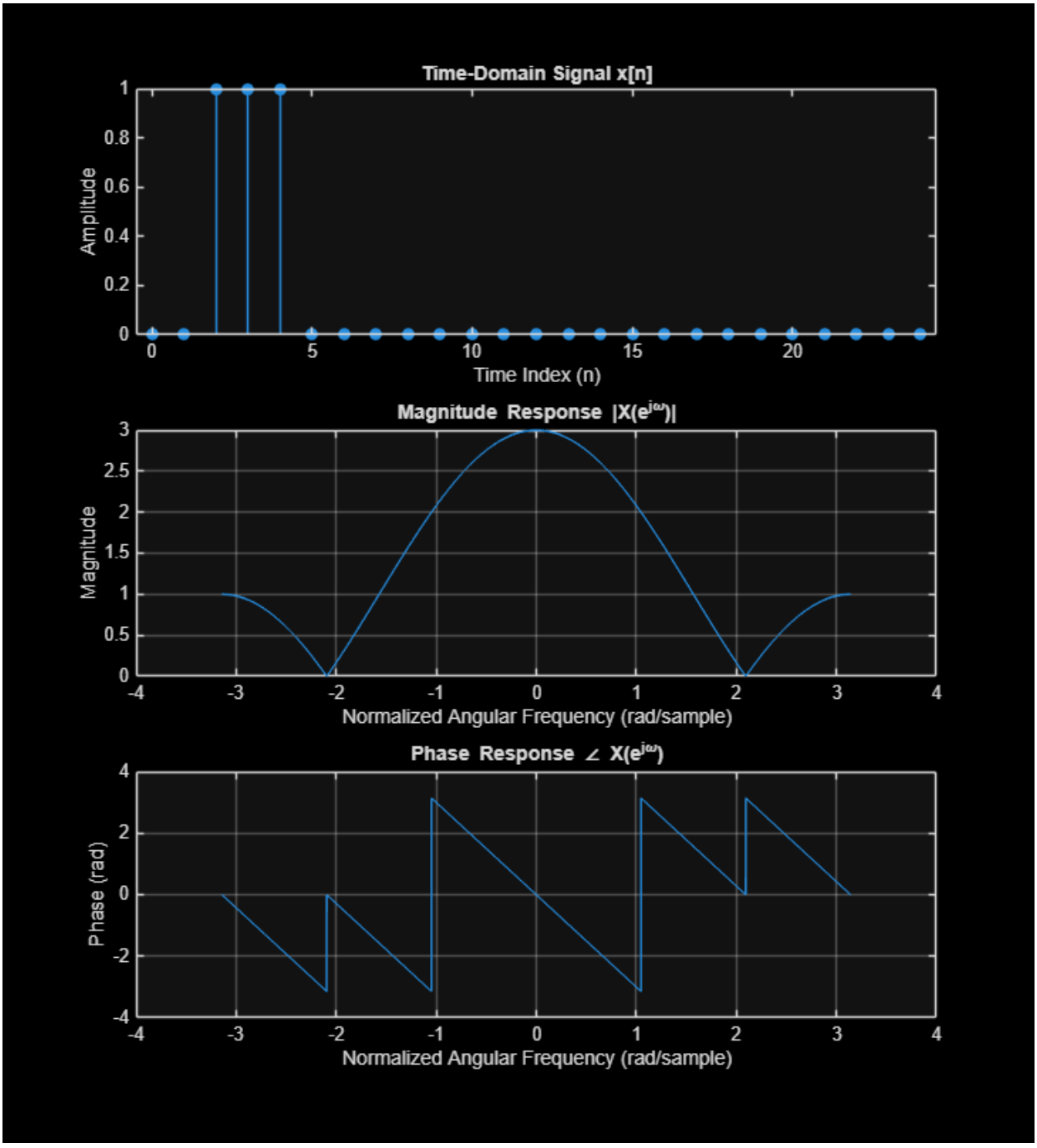
```
% ------------------------------------------------------------------
% x[n] = (-0.9)^n u[n]

n = 0:24;                       % required sample range

% Construct x[n] (exponentially decaying sequence)
x = (-0.9).^n;

% Frequency vector for DTFT
w = -pi:pi/5000:pi;

% Compute DTFT of x[n]
X = DTFT(x(:), w);

% -------------------- Plots --------------------
figure

% Time-domain signal
subplot(3,1,1)
stem(n, x, 'filled')
xlim([-0.5 24.5])
xlabel('Time Index (n)')
ylabel('Amplitude')
title('Time-Domain Signal x[n]')

% Magnitude spectrum
subplot(3,1,2)
plot(w, abs(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude Response |X(e^{j\omega})|')

% Phase spectrum
subplot(3,1,3)
plot(w, angle(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Phase (rad)')
title('Phase Response \angle X(e^{j\omega})')

% Visual classification
disp('2(c): predominantly HIGH frequency (peaks at + or - pi rad)')

% ------------------------------------------------------------------
% 2(d)
% ------------------------------------------------------------------
%  x[n] = (-0.9)^n cos((3*pi/4)n) u[n]
n = 0:24;                       % required sample range

% Construct x[n]
x = (-0.9).^n .* cos((3*pi/4).*n);

% Frequency vector for DTFT
```

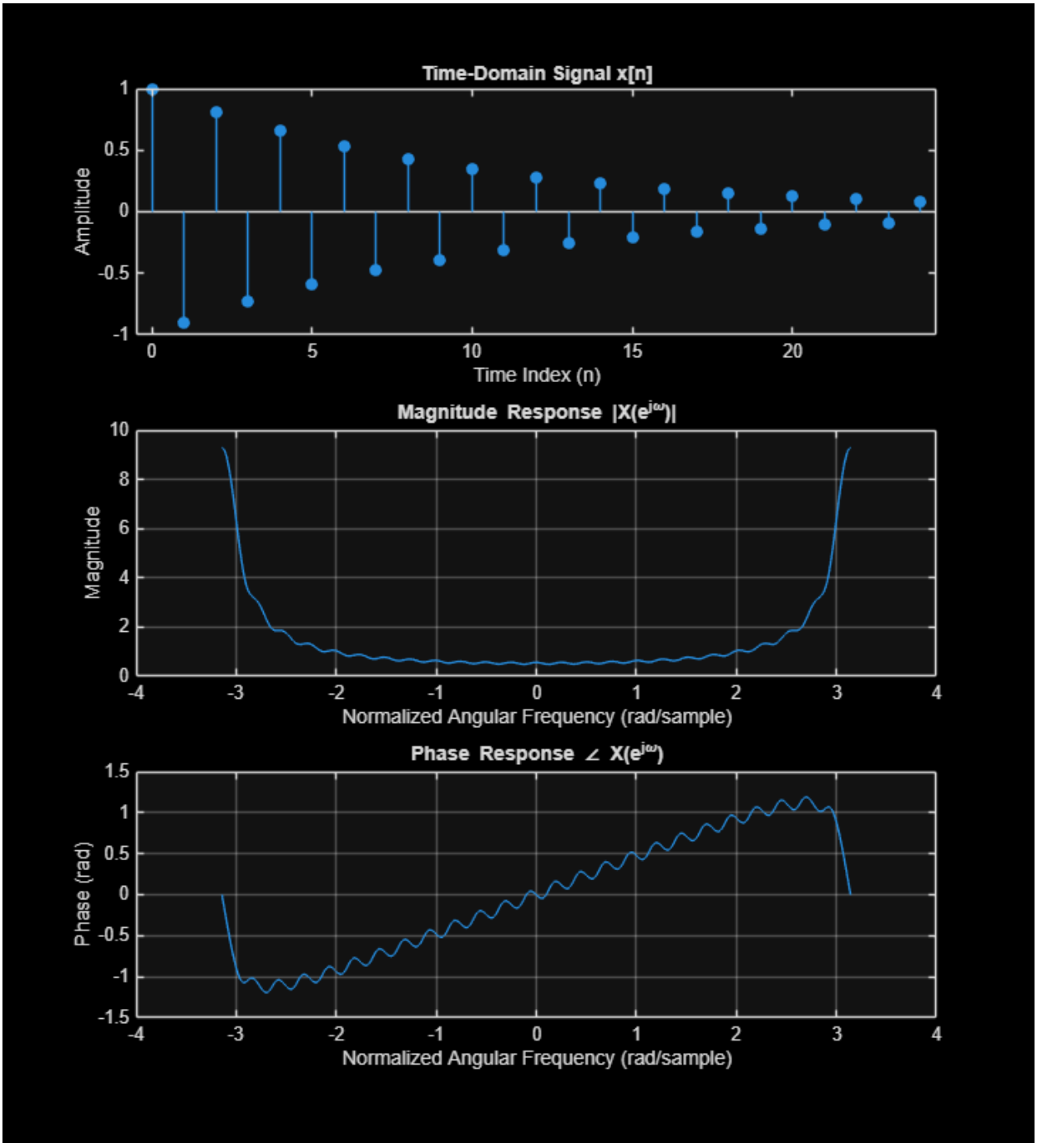
```
w = -pi:pi/5000:pi;

% Compute DTFT of x[n]
X = DTFT(x(:), w);

% -------------------- Plots --------------------
figure

% Time-domain signal
subplot(3,1,1)
stem(n, x, 'filled')
xlim([-0.5 24.5])
xlabel('Time Index (n)')
ylabel('Amplitude')
title('Time-Domain Signal x[n]')

% Magnitude spectrum
subplot(3,1,2)
plot(w, abs(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude Response |X(e^{j\omega})|')

% Phase spectrum
subplot(3,1,3)
plot(w, angle(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Phase (rad)')
title('Phase Response \angle X(e^{j\omega})')

% Visual classification
disp('2(d): predominantly NEITHER (closer to low - peaks around + or - 0.8
rad')


% ------------------------------------------------------------------
% 2(e)
% ------------------------------------------------------------------
% x[n] = (-0.9)^n cos((pi/4)n) u[n]

n = 0:24;                       % required sample range

% Construct x[n]
x = (-0.9).^n .* cos((pi/4).*n);

% Frequency vector for DTFT
w = -pi:pi/5000:pi;

% Compute DTFT of x[n]
X = DTFT(x(:), w);

% -------------------- Plots --------------------
```

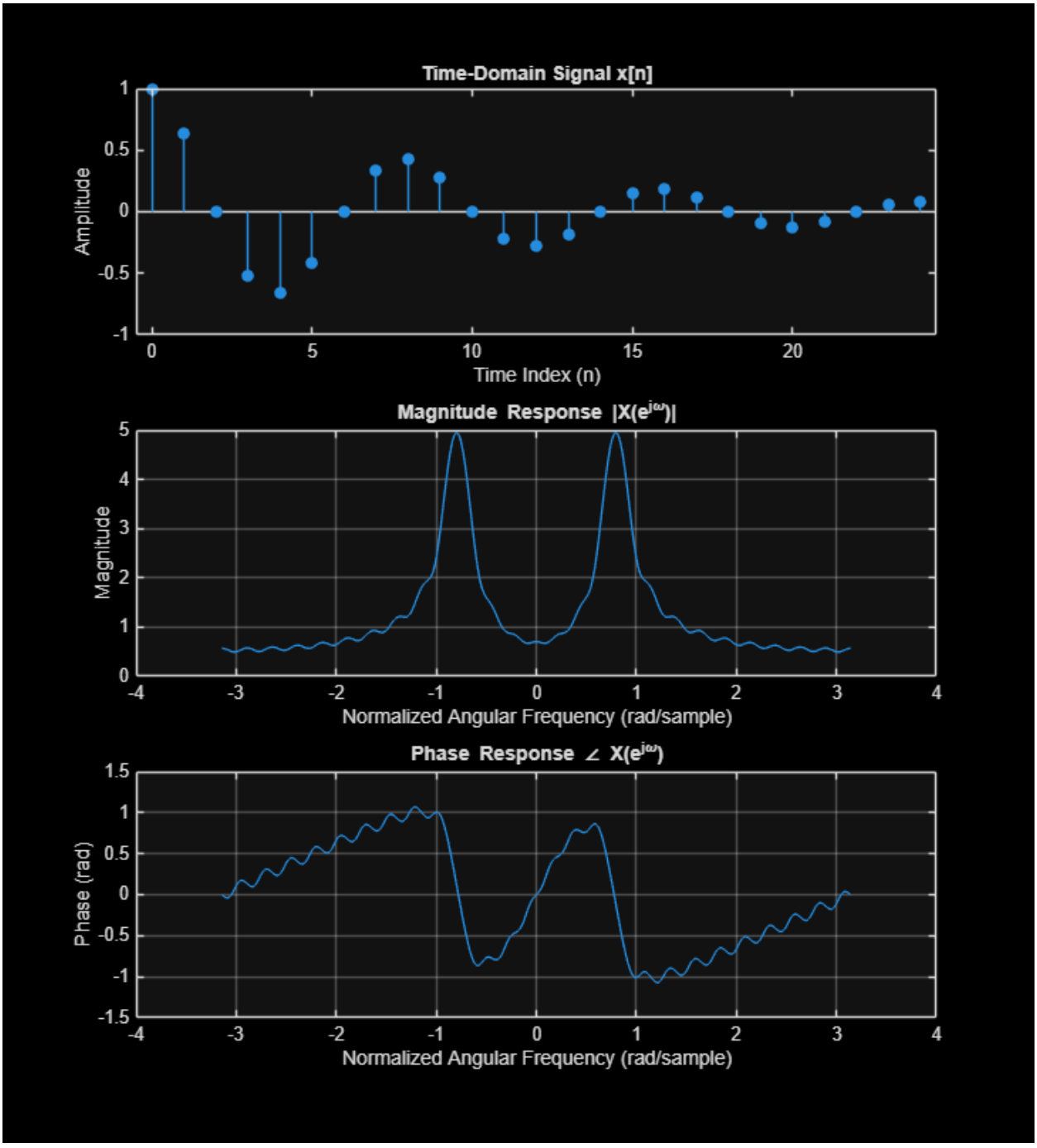
```
figure

% Time-domain signal
subplot(3,1,1)
stem(n, x, 'filled')
xlim([-0.5 24.5])
xlabel('Time Index (n)')
ylabel('Amplitude')
title('Time-Domain Signal x[n]')

% Magnitude spectrum
subplot(3,1,2)
plot(w, abs(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude Response |X(e^{j\omega})|')

% Phase spectrum
subplot(3,1,3)
plot(w, angle(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Phase (rad)')
title('Phase Response \angle X(e^{j\omega})')

% Visual classification
disp('2(e): predominantly HIGH frequency (peaks around + or - 2.3 rad: I
could also see this as neither, but did not know which to put.')


% ------------------------------------------------------------------
% 2(f)
% ------------------------------------------------------------------

% x[n] = (-1)^n u[n]

n = 0:24;                       % required sample range

% Construct x[n]
x = (-1).^n;

% Frequency vector for DTFT
w = -pi:pi/5000:pi;

% Compute DTFT of x[n]
X = DTFT(x(:), w);

% -------------------- Plots --------------------
figure

% Time-domain signal
subplot(3,1,1)
stem(n, x, 'filled')
```

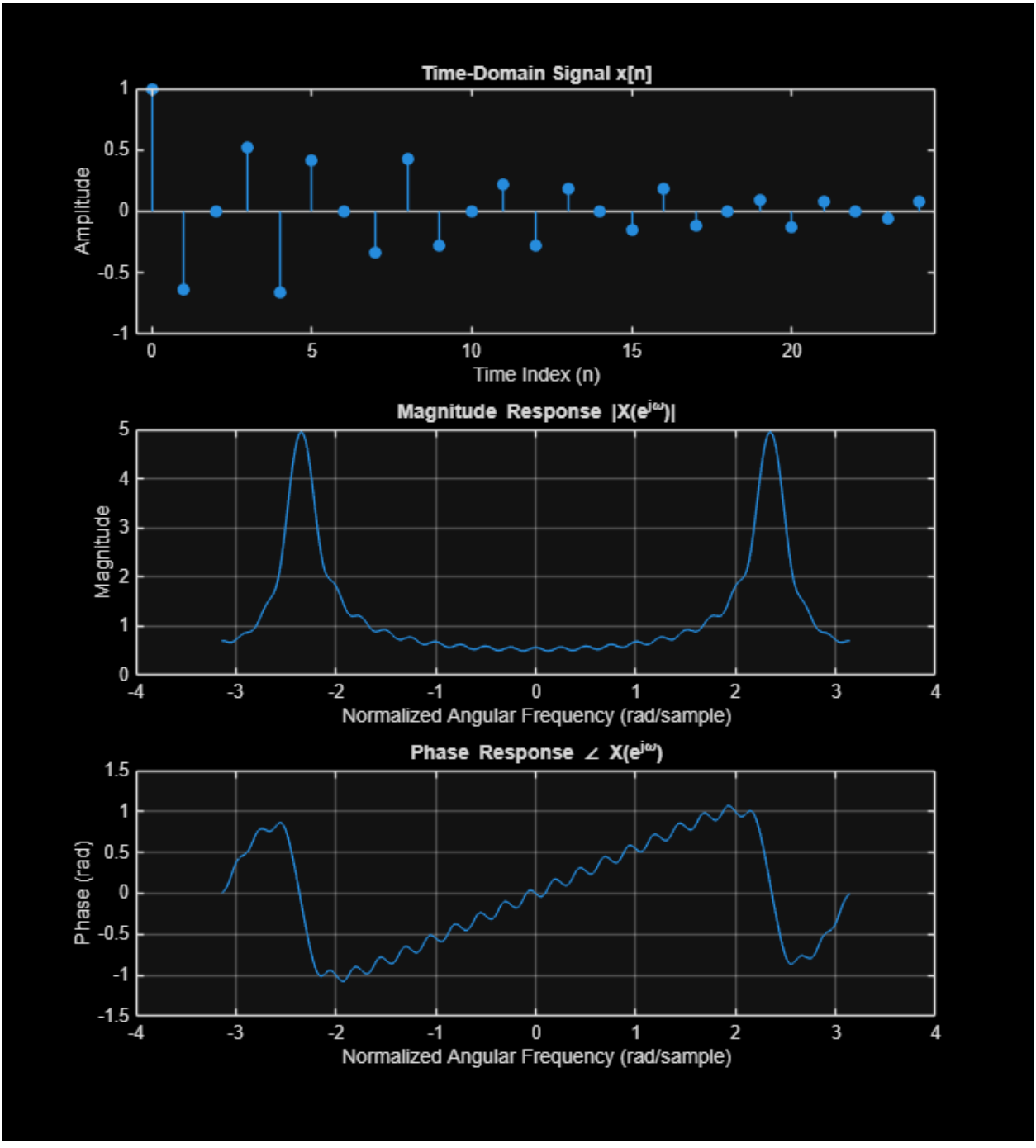
```
xlim([-0.5 24.5])
xlabel('Time Index (n)')
ylabel('Amplitude')
title('Time-Domain Signal x[n]')

% Magnitude spectrum
subplot(3,1,2)
plot(w, abs(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude Response |X(e^{j\omega})|')

% Phase spectrum
subplot(3,1,3)
plot(w, angle(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Phase (rad)')
title('Phase Response \angle X(e^{j\omega})')

% Visual classification
disp('2(f): predominantly HIGH frequency (peaks at + or - pi rad)')
```

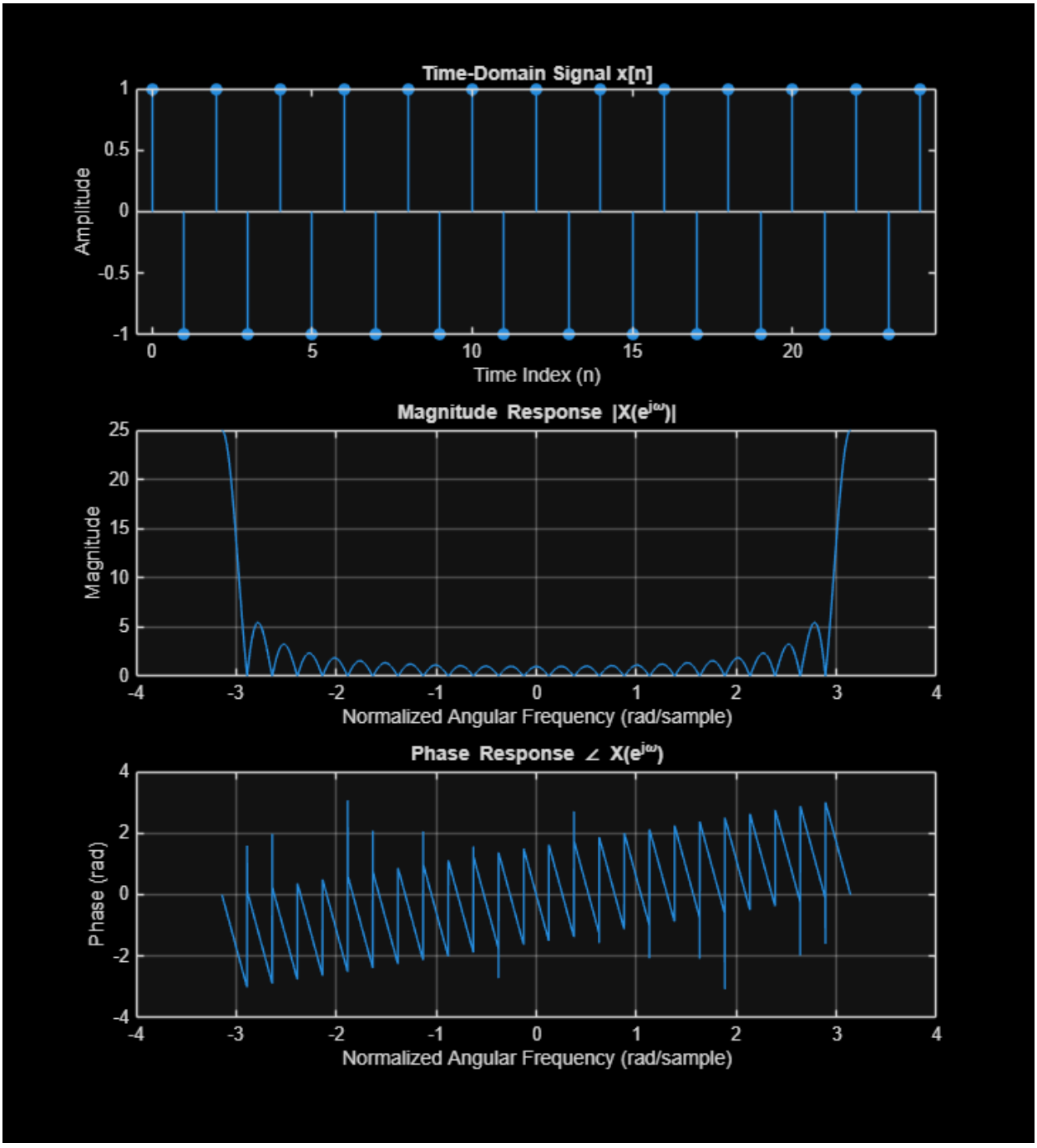

Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)


Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)


Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)


Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)


Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)


Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)

==========

==========

3. DTFT PROPERTIES

==========

==========

```
% y[n] = 4x[n]


n = 0:24;                       % required sample range

% Construct x[n] = [(25/46) - (21/46)cos((pi/6)n)] (u[n] - u[n-12])
x = ((25/46) - (21/46)*cos((pi/6).*n)) .* (n <= 11);

% Construct y[n]
y = 4*x;

% Frequency vector for DTFT
w = -pi:pi/5000:pi;

% Compute DTFT of y[n]
Y = DTFT(y(:), w);

% -------------------- Plots --------------------
figure

% Time-domain signal
subplot(3,1,1)
stem(n, y, 'filled')
xlim([-0.5 24.5])
xlabel('Time Index (n)')
ylabel('Amplitude')
title('Time-Domain Signal y[n]')

% Magnitude spectrum
subplot(3,1,2)
plot(w, abs(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude Response |Y(e^{j\omega})|')

% Phase spectrum
subplot(3,1,3)
plot(w, angle(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
```

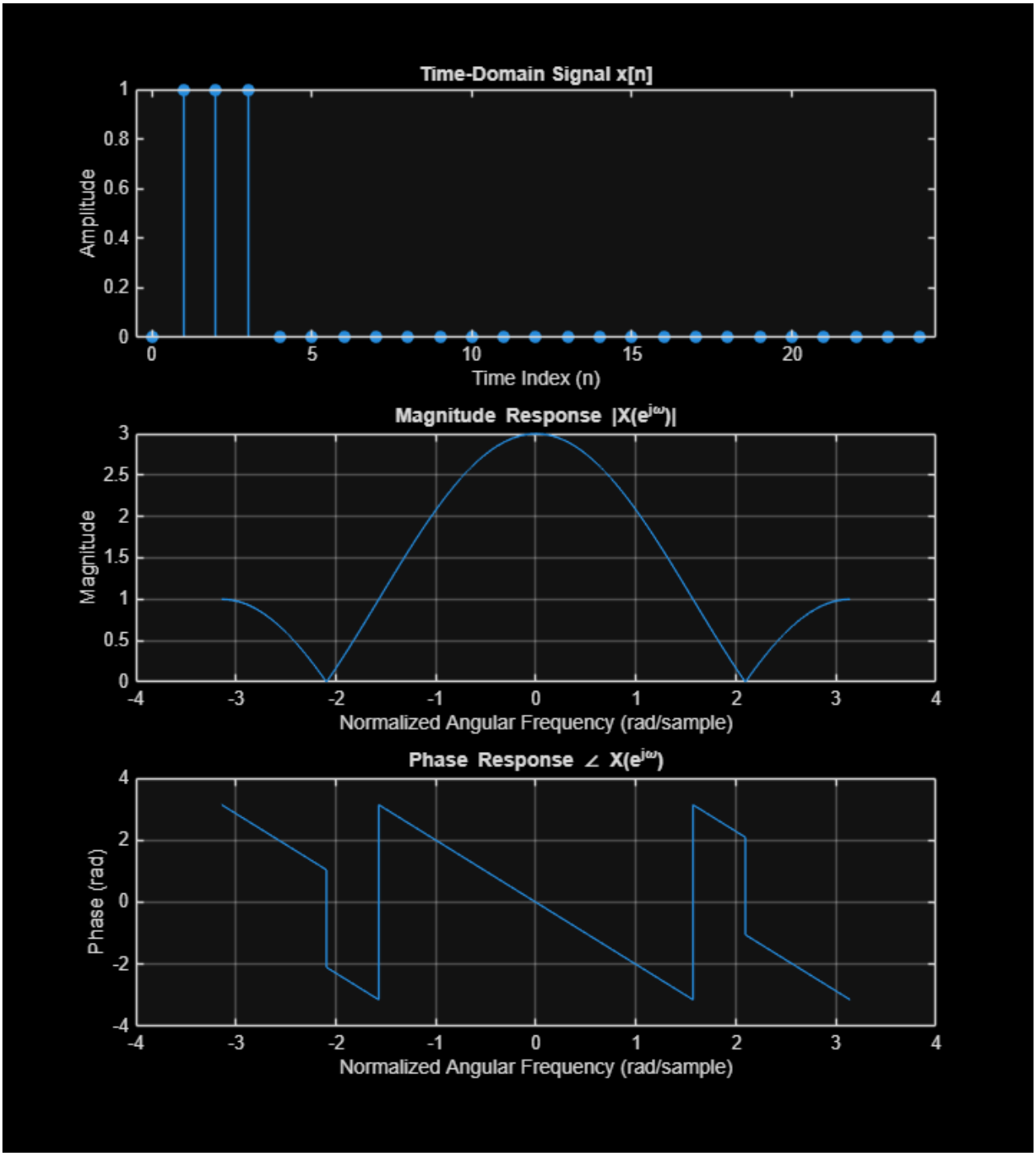
```
ylabel('Phase (rad)')
title('Phase Response \angle Y(e^{j\omega})')



% ------------------------------------------------------------------
% 3(b)
% ------------------------------------------------------------------

% y[n] = 4x[n-8]

n = 0:24;                       % required sample range

% Construct x[n]
x = ((25/46) - (21/46)*cos((pi/6).*n)) .* (n <= 11);

% Time-shifted & scaled signal
y = 4 * (((25/46) - (21/46)*cos((pi/6).*(n-8))) .* ((n-8) >= 0 & (n-8) <=
11));

% Frequency vector for DTFT
w = -pi:pi/5000:pi;

% Compute DTFT
Y = DTFT(y(:), w);

% -------------------- Plots --------------------
figure

subplot(3,1,1)
stem(n, y, 'filled')
xlim([-0.5 24.5])
xlabel('Time Index (n)')
ylabel('Amplitude')
title('Time-Domain Signal y[n]')

subplot(3,1,2)
plot(w, abs(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude Response |Y(e^{j\omega})|')

subplot(3,1,3)
plot(w, angle(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Phase (rad)')
title('Phase Response \angle Y(e^{j\omega})')


% ------------------------------------------------------------------
% 3(c)
```

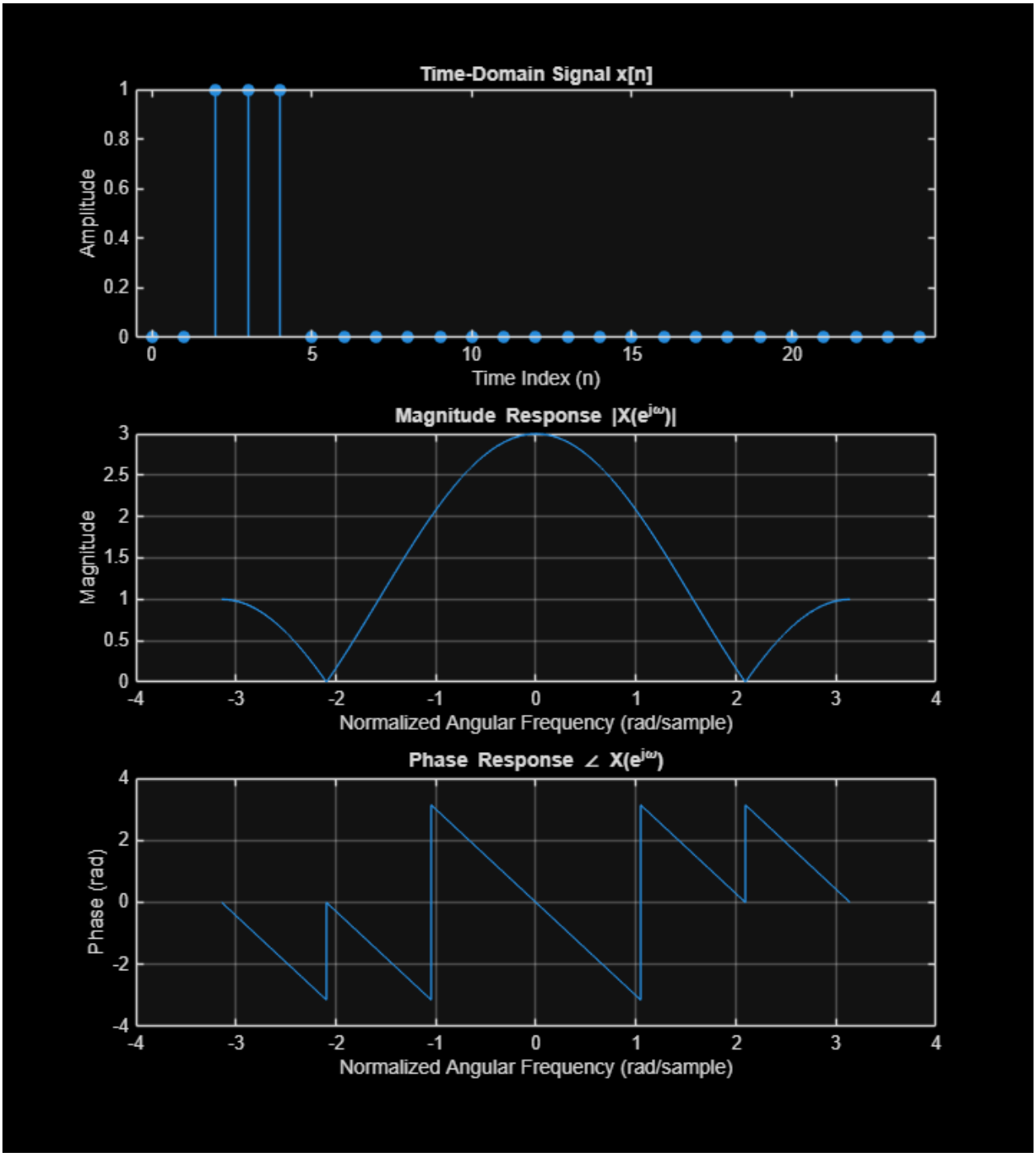
```
% ------------------------------------------------------------------


% y[n] = x[n] cos((pi/4)n)

n = 0:24;                       % required sample range

% Construct x[n]
x = ((25/46) - (21/46)*cos((pi/6).*n)) .* (n <= 11);

% Modulated signal
y = x .* cos((pi/4).*n);

% Frequency vector for DTFT
w = -pi:pi/5000:pi;

% Compute DTFT
Y = DTFT(y(:), w);

% -------------------- Plots --------------------
figure

subplot(3,1,1)
stem(n, y, 'filled')
xlim([-0.5 24.5])
xlabel('Time Index (n)')
ylabel('Amplitude')
title('Time-Domain Signal y[n]')

subplot(3,1,2)
plot(w, abs(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude Response |Y(e^{j\omega})|')

subplot(3,1,3)
plot(w, angle(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Phase (rad)')
title('Phase Response \angle Y(e^{j\omega})')


% ------------------------------------------------------------------
% 3(d)
% ------------------------------------------------------------------

% % y[n] = x[n] * x[n]
%
% n = 0:24;                       % required sample range
%
% % Construct x[n]
% x = ((25/46) - (21/46)*cos((pi/6).*n)) .* (n <= 11);
```

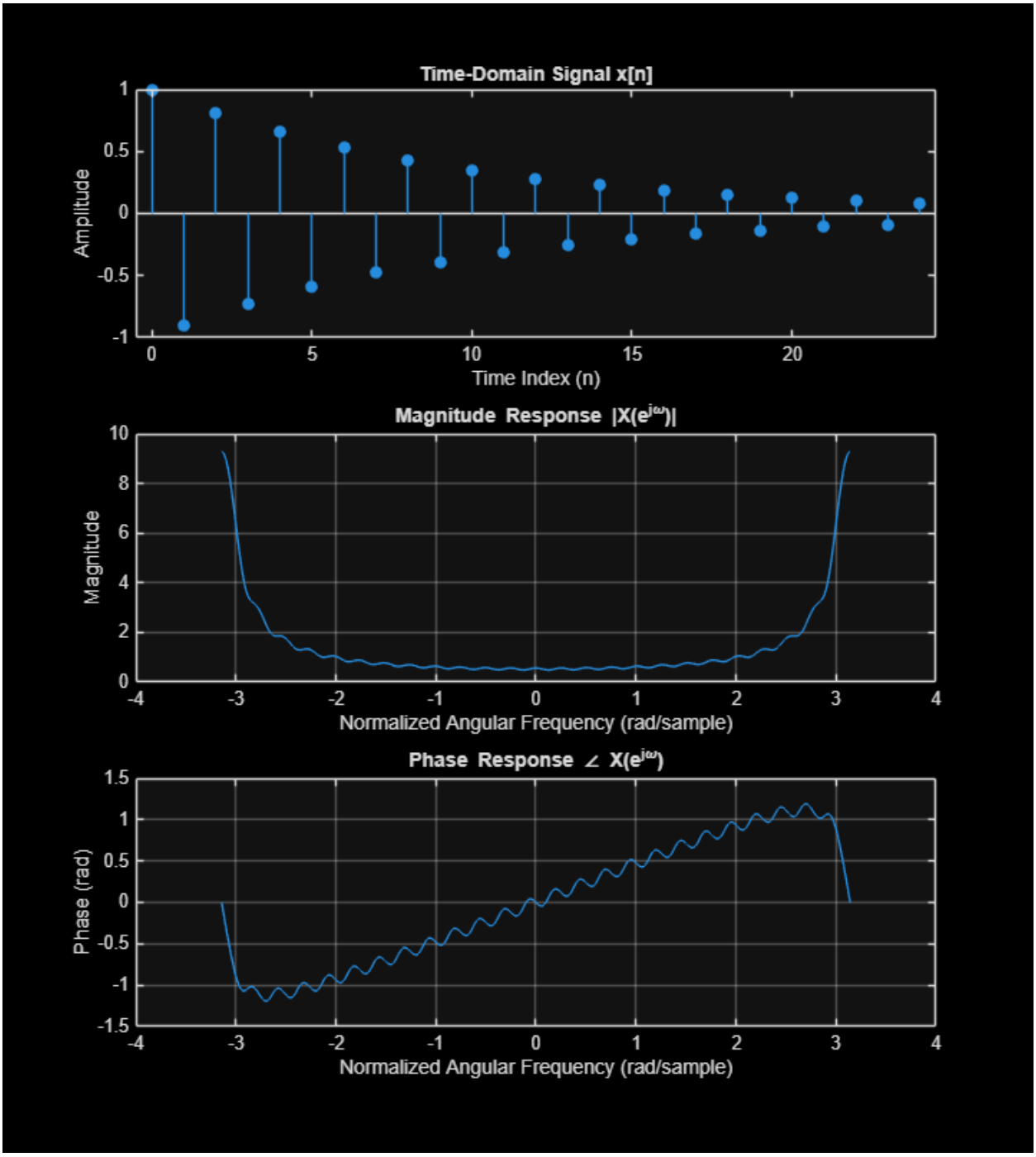
```
%
% % Convolution y[n] = x[n] * x[n]
% y_full = conv(x, x);          % length = 49 (0 to 48)
% y = y_full(1:25);             % keep n = 0..24 for plotting/DTFT (I was
not sure if I should truncate before doing the dtft or after)
%
% % Frequency vector for DTFT
% w = -pi:pi/5000:pi;
%
% % Compute DTFT of y[n]
% Y = DTFT(y(:), w);
%
% % -------------------- Plots --------------------
% figure
%
% subplot(3,1,1)
% stem(n, y, 'filled')
% xlim([-0.5 24.5])
% xlabel('Time Index (n)')
% ylabel('Amplitude')
% title('Time-Domain Signal y[n]')
%
% subplot(3,1,2)
% plot(w, abs(Y))
% grid on
% xlabel('Normalized Angular Frequency (rad/sample)')
% ylabel('Magnitude')
% title('Magnitude Response |Y(e^{j\omega})|')
%
% subplot(3,1,3)
% plot(w, angle(Y))
% grid on
% xlabel('Normalized Angular Frequency (rad/sample)')
% ylabel('Phase (rad)')
% title('Phase Response \angle Y(e^{j\omega})')


% I copied the code above and made changes to not truncate before the dtft,
% but the results looked the exact same.

n = 0:24;
% Construct x[n]
x = ((25/46) - (21/46)*cos((pi/6).*n)) .* (n <= 11);
% Convolution y[n] = x[n] * x[n]
y_full = conv(x, x);          % length = 49 (0 to 48)
% Frequency vector for DTFT
w = -pi:pi/5000:pi;
% Compute DTFT of full convolution result
Y = DTFT(y_full(:), w);
% -------------------- Plots --------------------
figure
subplot(3,1,1)
stem(n, y_full(1:25), 'filled')
xlim([-0.5 24.5])
```

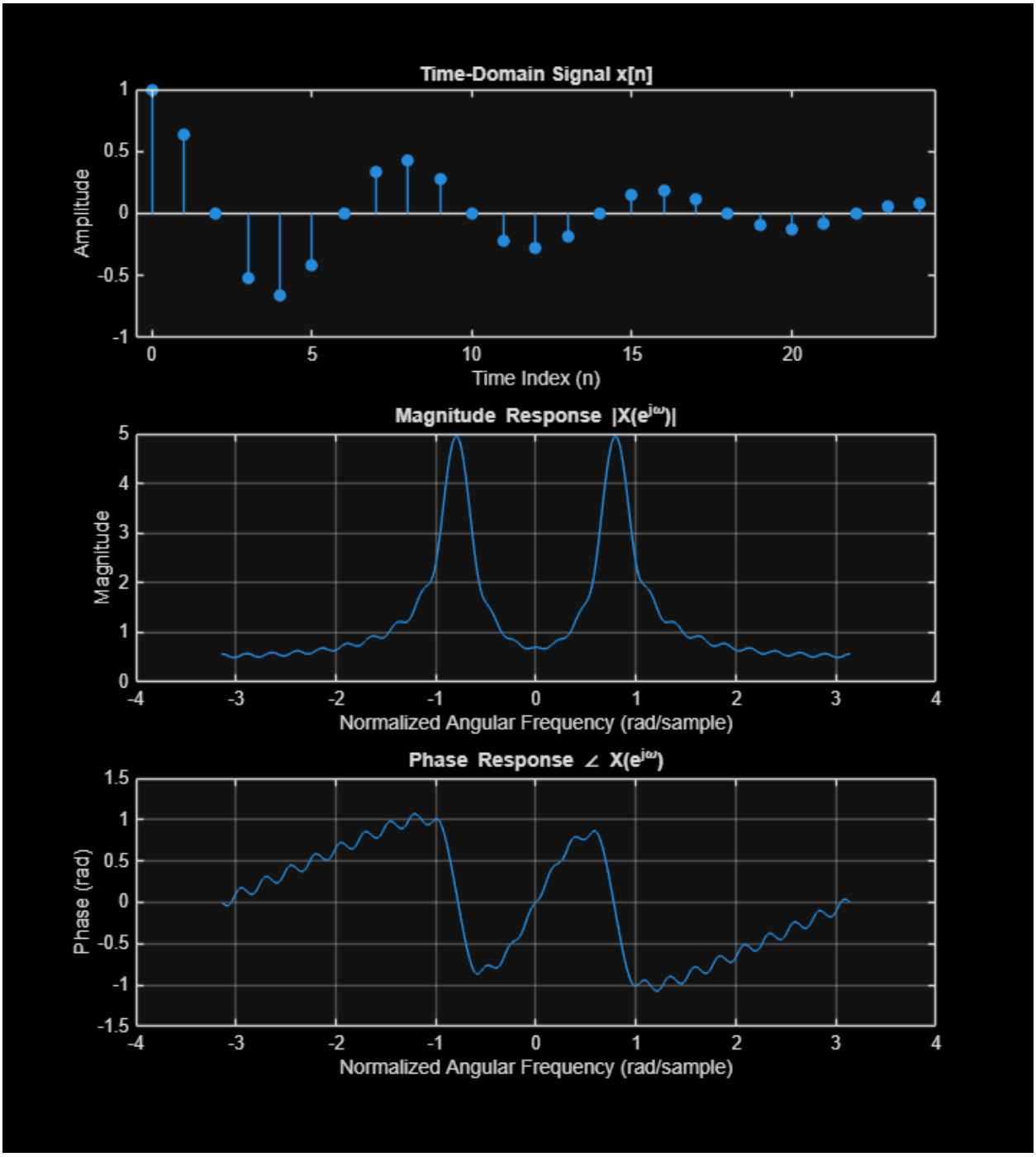
```
xlabel('Time Index (n)')
ylabel('Amplitude')
title('Time-Domain Signal y[n]')
subplot(3,1,2)
plot(w, abs(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude Response |Y(e^{j\omega})|')
subplot(3,1,3)
plot(w, angle(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Phase (rad)')
title('Phase Response \angle Y(e^{j\omega})')


% ------------------------------------------------------------------
% 3(e)
% ------------------------------------------------------------------


 % y[n] = |x[n]|^2

n = 0:24;                       % required sample range

% Construct x[n]
x = ((25/46) - (21/46)*cos((pi/6).*n)) .* (n <= 11);

% Nonlinear operation
y = abs(x).^2;

% Frequency vector for DTFT
w = -pi:pi/5000:pi;

% Compute DTFT
Y = DTFT(y(:), w);

% -------------------- Plots --------------------
figure

subplot(3,1,1)
stem(n, y, 'filled')
xlim([-0.5 24.5])
xlabel('Time Index (n)')
ylabel('Amplitude')
title('Time-Domain Signal y[n]')

subplot(3,1,2)
plot(w, abs(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Magnitude')
title('Magnitude Response |Y(e^{j\omega})|')
```

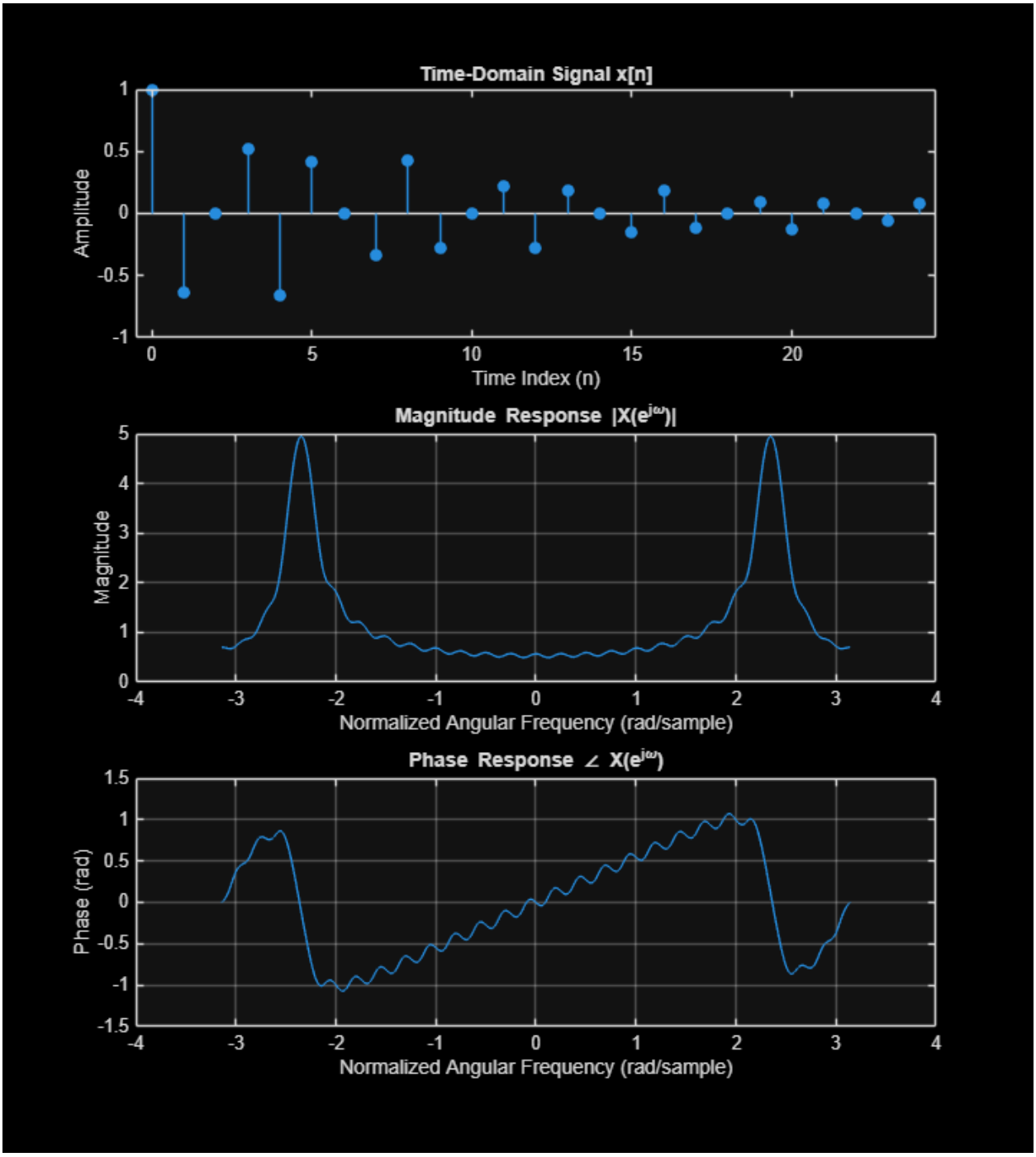
```
subplot(3,1,3)
plot(w, angle(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('Phase (rad)')
title('Phase Response \angle Y(e^{j\omega})')
```

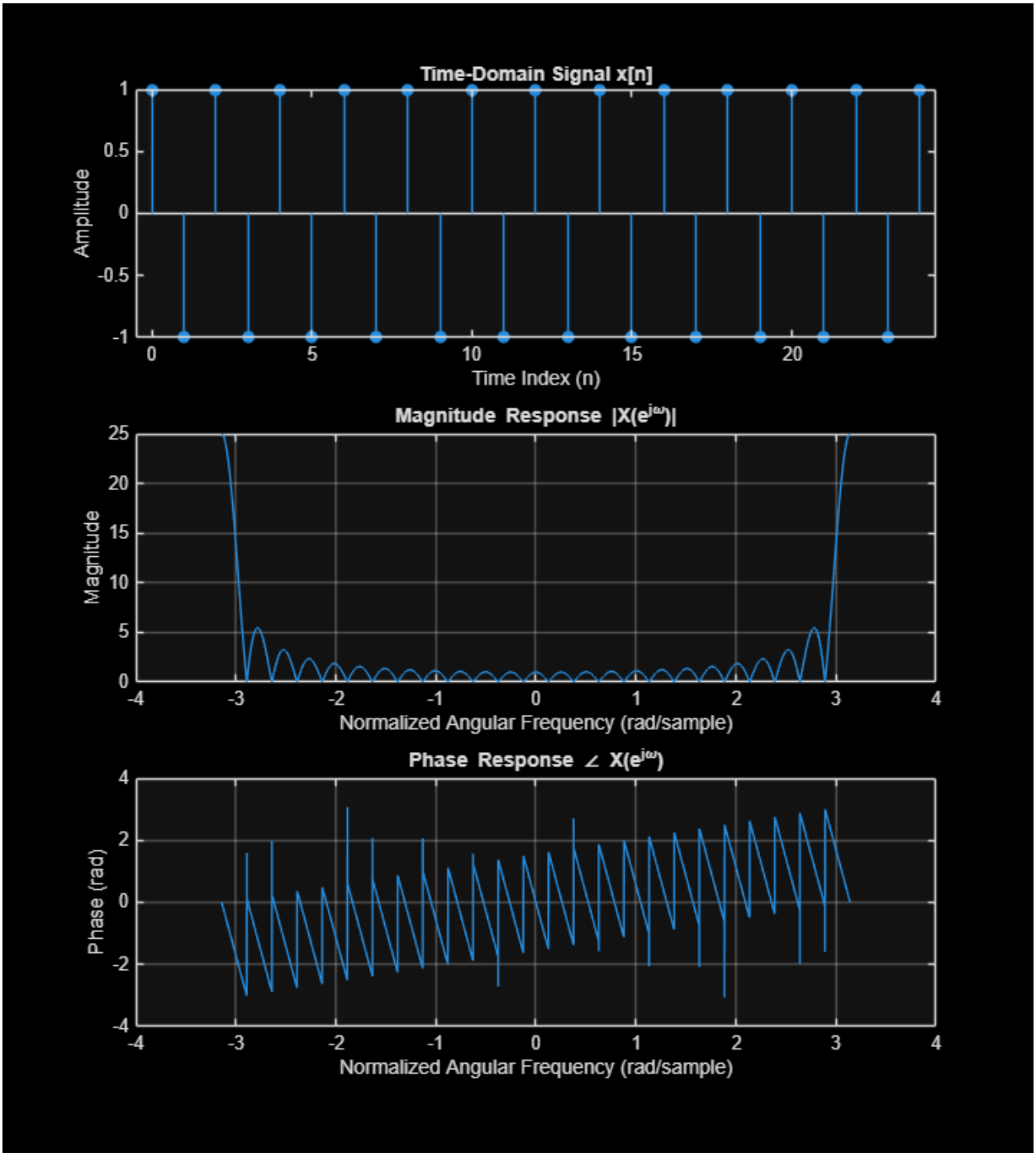


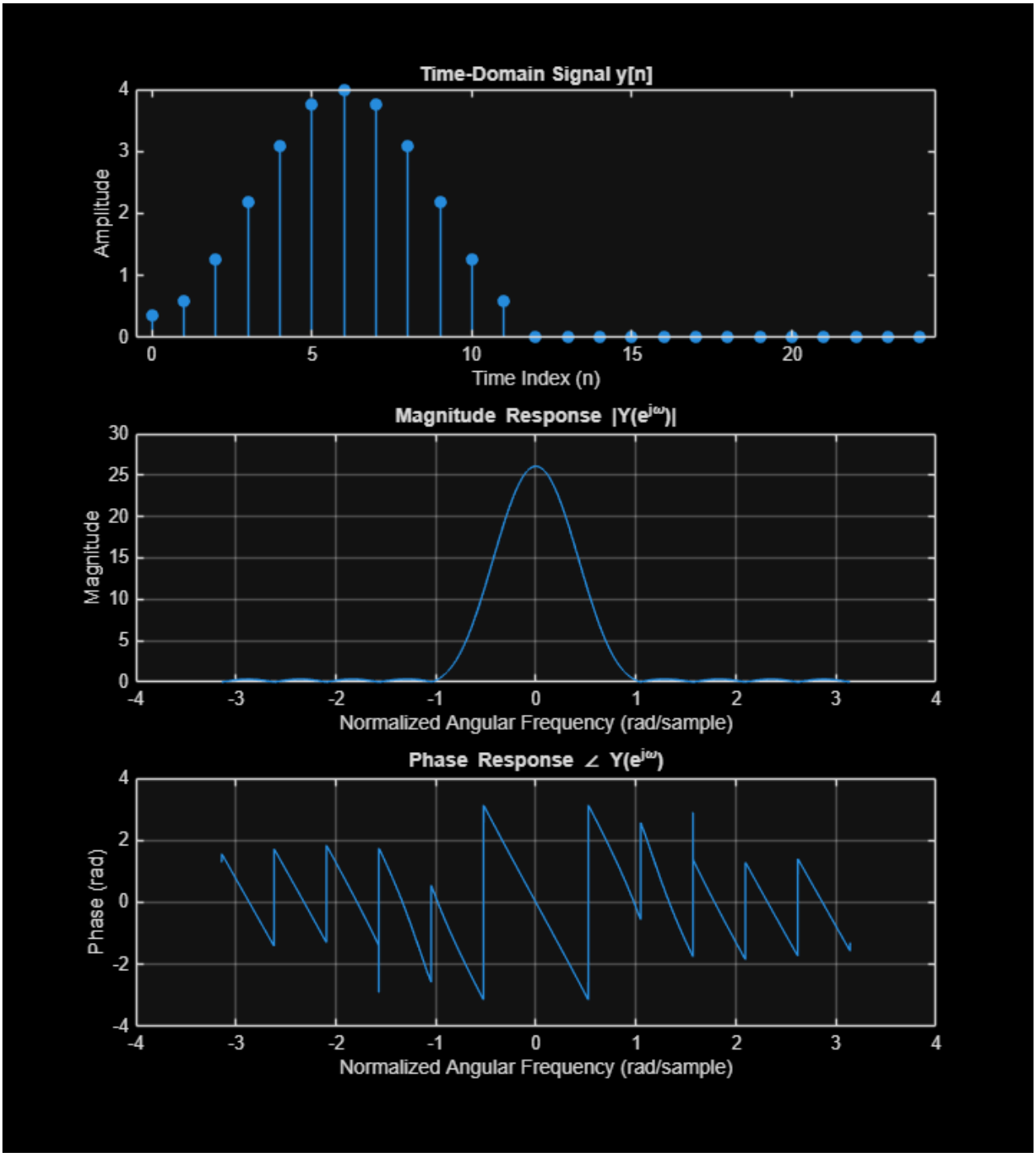

Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)

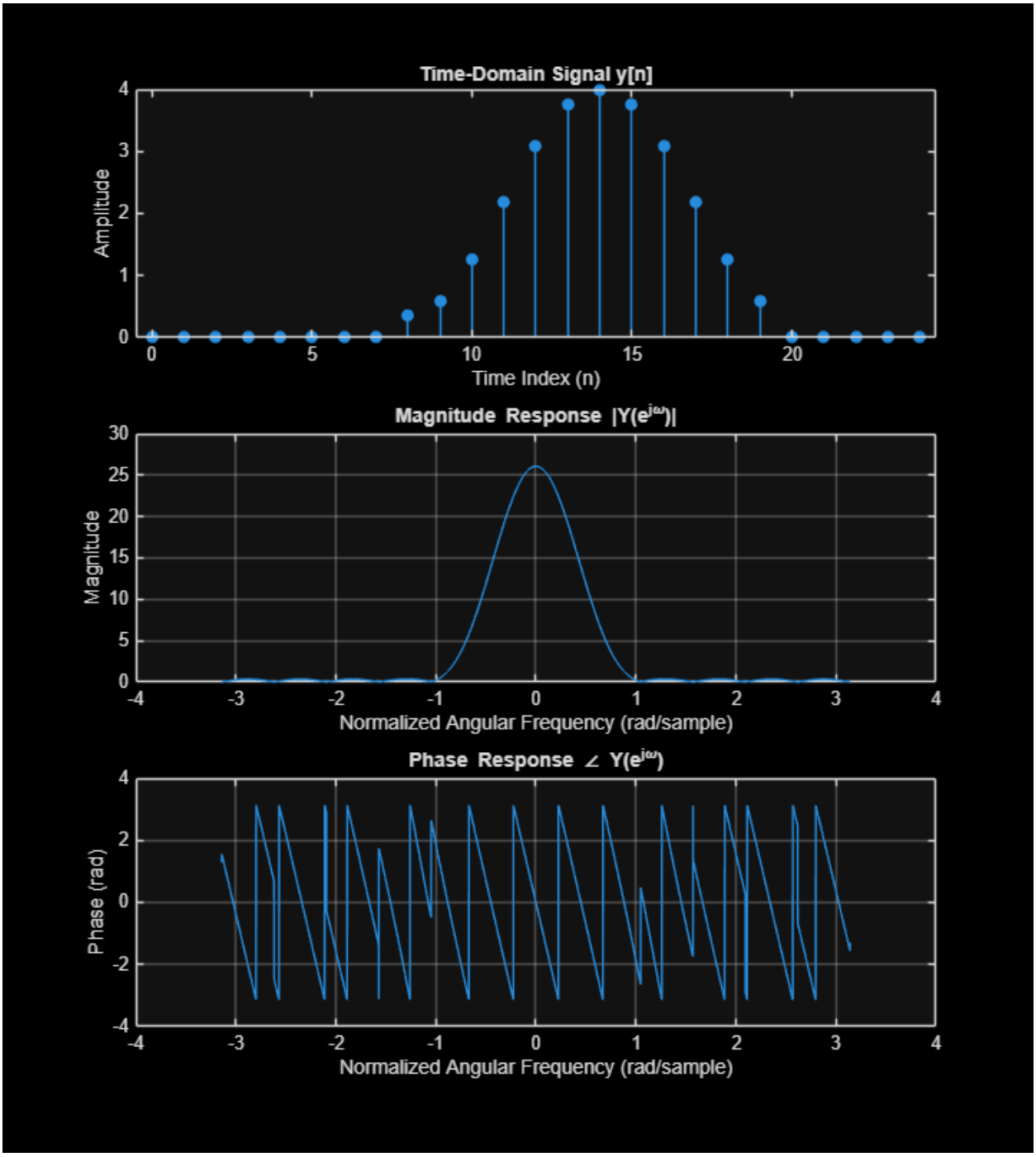

Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)

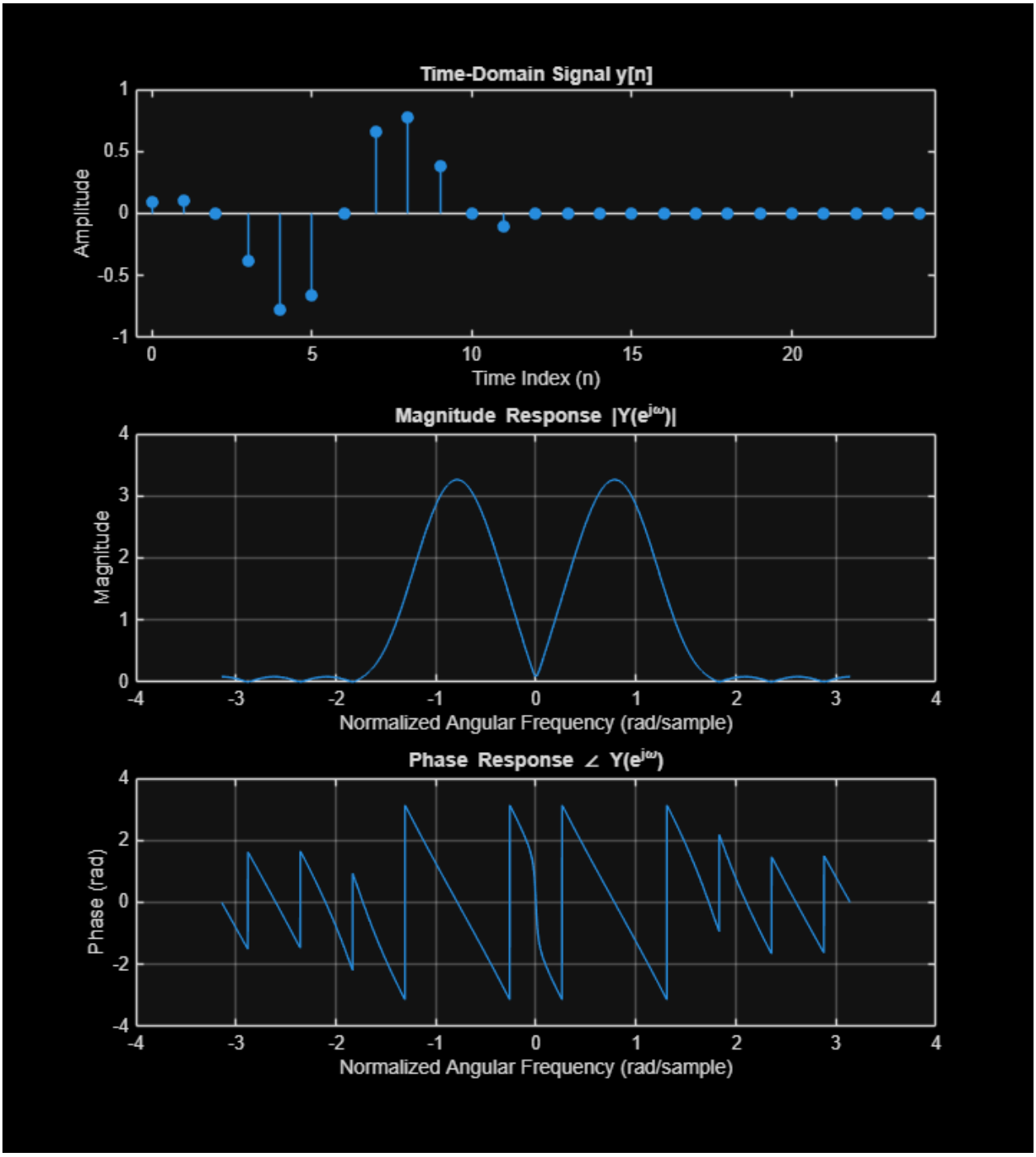

Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)

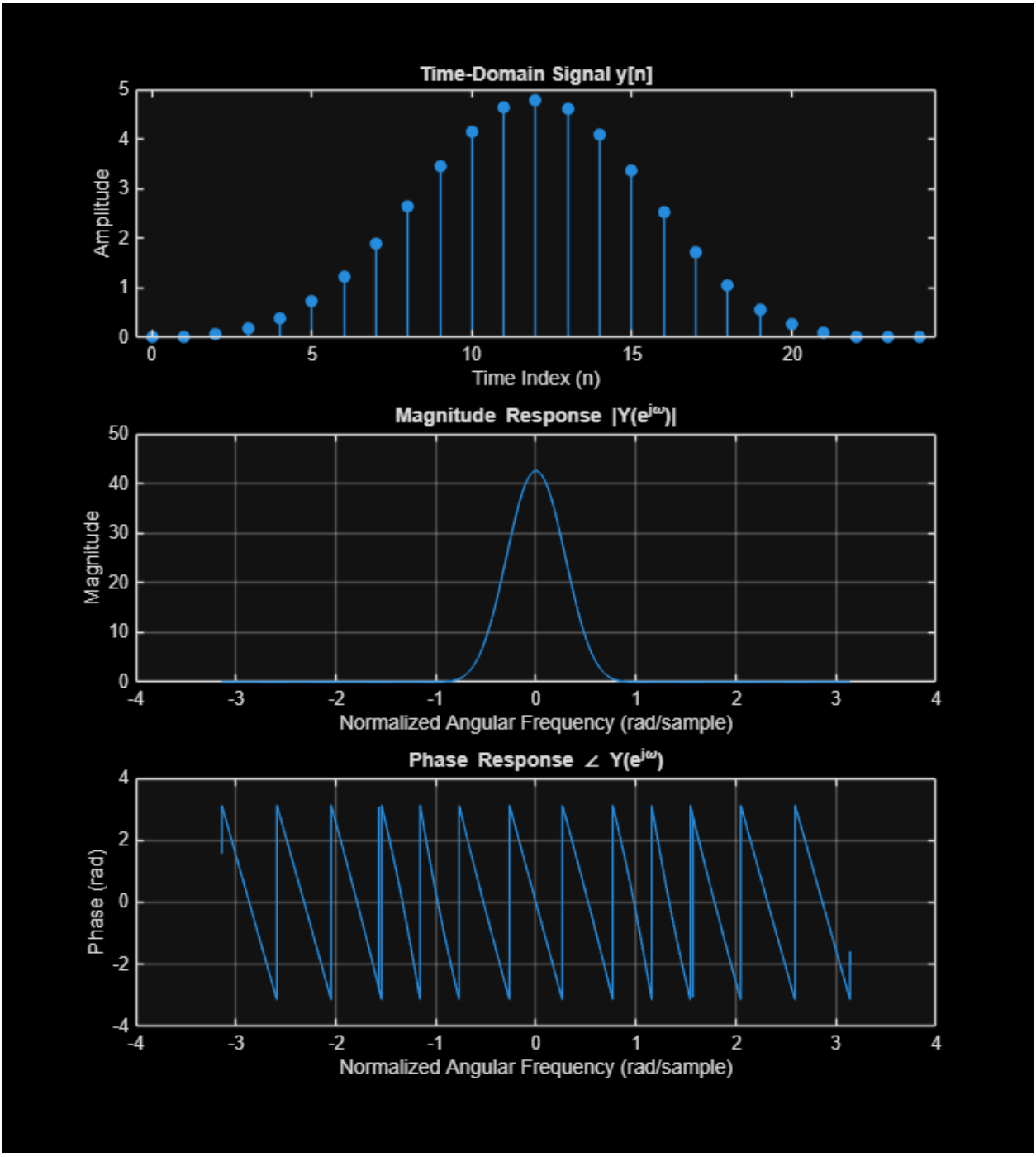

Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)

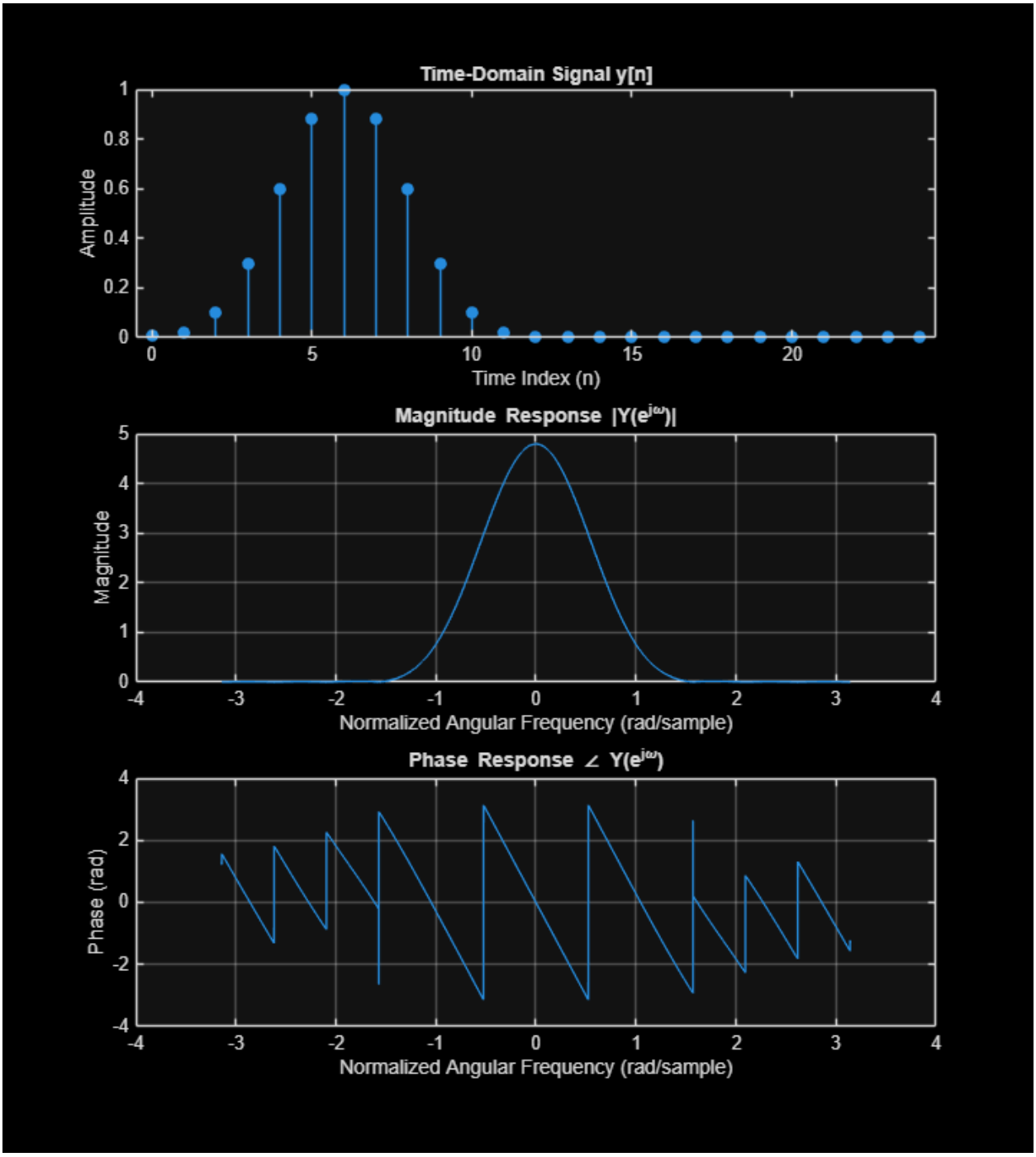

Time-Domain Signal x[n]
Amplitude
Time Index (n)
Magnitude Response |X(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ X(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)


Time-Domain Signal y[n]
Amplitude
Time Index (n)
Magnitude Response |Y(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ Y(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)


Time-Domain Signal y[n]
Amplitude
Time Index (n)
Magnitude Response |Y(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ Y(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)


Time-Domain Signal y[n]
Amplitude
Time Index (n)
Magnitude Response |Y(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ Y(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)


Time-Domain Signal y[n]
Amplitude
Time Index (n)
Magnitude Response |Y(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ Y(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)


Time-Domain Signal y[n]
Amplitude
Time Index (n)
Magnitude Response |Y(e^jω)|
Magnitude
Normalized Angular Frequency (rad/sample)
Phase Response ∠ Y(e^jω)
Phase (rad)
Normalized Angular Frequency (rad/sample)

==========

==========

4. NULLING FILTER

==========

==========

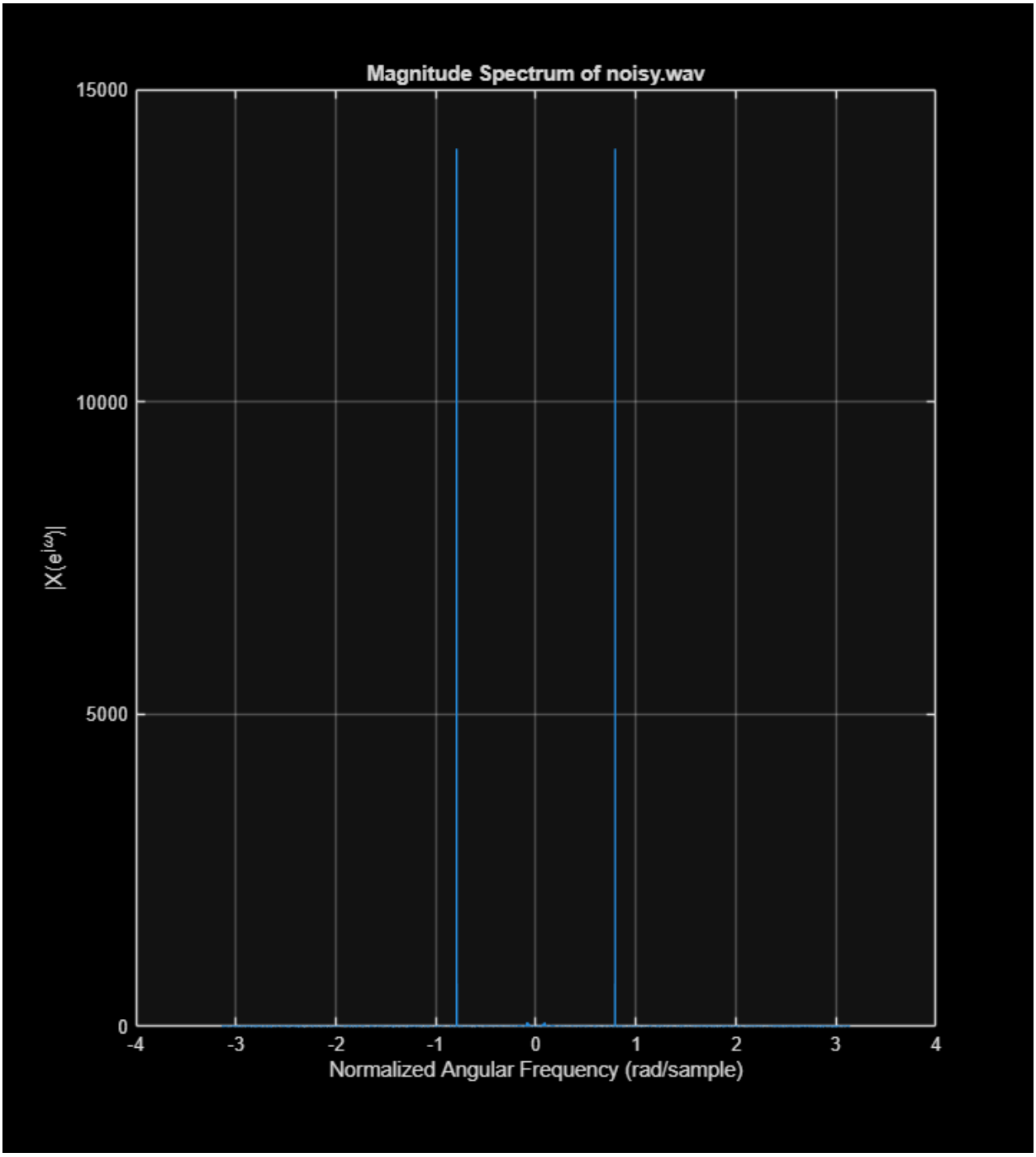
```
% MAKE SURE FILES BELOW ARE IN SAME DIRECTORY AS THIS FILE
[x1, fs] = audioread('noisy.wav');
[x2, fs] = audioread('noisy2.wav');



% ------------------------------------------------------------------
% 4(a)
% ------------------------------------------------------------------
x = x1(:,1);                    % use one channel
N = length(x);

w = -pi:pi/10000:pi;            % > 10,000 points
X = DTFT(x, w);

figure
plot(w, abs(X))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('|X(e^{j\omega})|')
title('Magnitude Spectrum of noisy.wav')


% ------------------------------------------------------------------
% 4(b)
% ------------------------------------------------------------------
[~, idx] = max(abs(X));

w0 = abs(w(idx));               % ensure positive frequency
f0 = w0 * fs / (2*pi);

disp(['Noise normalized angular frequency: ', num2str(w0), ' rad/sample'])
disp(['Noise continuous-time cyclic frequency: ', num2str(f0), ' Hz'])
disp(fs);


% ------------------------------------------------------------------
% 4(c)
% ------------------------------------------------------------------
```

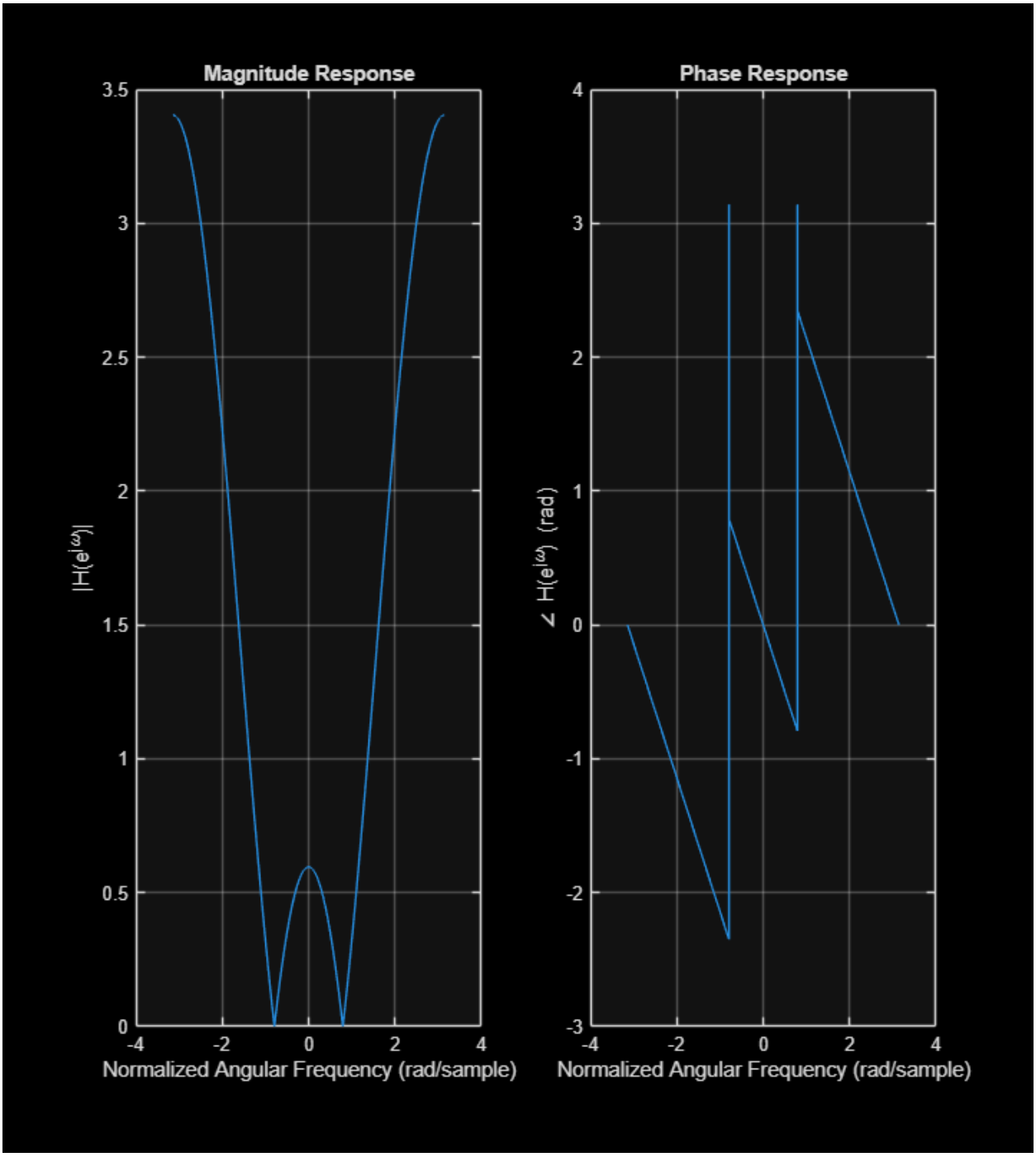
```
% w0 = 0.79262;                  % measured noise frequency (rad/sample)
% already computed

h = [1; -2*cos(w0); 1];          % impulse response of the nulling filter
n = 0:length(h)-1;

w = -pi:pi/10000:pi;             % lots of frequency points
H = DTFT(h, w);

figure
subplot(1,2,1)
plot(w, abs(H))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('|H(e^{j\omega})|')
title('Magnitude Response')

subplot(1,2,2)
plot(w, angle(H))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('\angle H(e^{j\omega}) (rad)')
title('Phase Response')


% ------------------------------------------------------------------
% 4(d)
% ------------------------------------------------------------------

%w0 = 0.79262;                   % chosen to remove noisy frequency (rad/
sample)
%already computed

% Nulling FIR impulse response
h = [1; -2*cos(w0); 1];

% Use one channel if stereo
x = x1(:,1);

% Apply filter via convolution
y = conv(x, h);

% DTFT (use >= 10,000 points)
w = -pi:pi/10000:pi;
Y = DTFT(y, w);

figure
subplot(1,2,1)
plot(w, abs(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('|Y(e^{j\omega})|')
title('Filtered noisy.wav Magnitude')
```

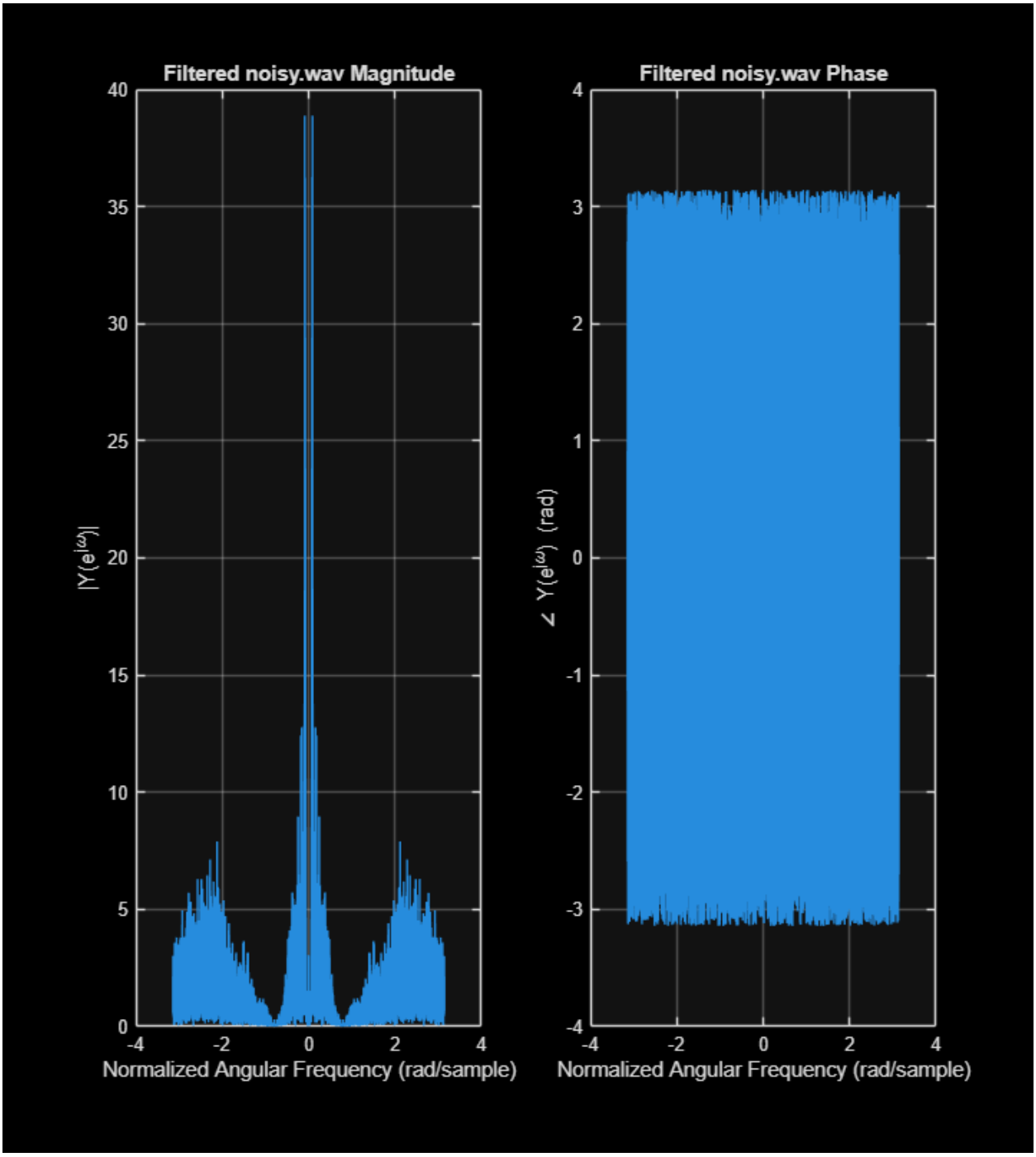
```
subplot(1,2,2)
plot(w, angle(Y))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('\angle Y(e^{j\omega}) (rad)')
title('Filtered noisy.wav Phase')


% ------------------------------------------------------------------
% 4(e)
% ------------------------------------------------------------------

%w0 = 0.79262;     already computed

h  = [1; -2*cos(w0); 1];

x = x1(:,1);                    % use one channel
y = conv(x, h);                 % apply nulling filter

% Normalize to avoid clipping when saving
y = y / max(abs(y));

audiowrite('denoised.wav', y, fs);
disp('Saved filtered audio as denoised.wav')


% ------------------------------------------------------------------
% 4(f)
% ------------------------------------------------------------------


x = x2(:,1);   % use one channel

% DTFT to find contaminated frequencies (>= 10000 points)
w = -pi:pi/10000:pi;
X2 = DTFT(x, w);

figure
plot(w, abs(X2))
grid on
xlabel('Normalized Angular Frequency (rad/sample)')
ylabel('|X(e^{j\omega})|')
title('Magnitude Spectrum of noisy2.wav')


% Manually identified noise frequencies from spectrum
w_noise = [0.376991; 0.753982; 1.50796];

disp('Contaminated normalized frequencies (rad/sample):')
disp(w_noise.')
disp('Contaminated frequencies (Hz):')
disp((w_noise * fs/(2*pi)).')

% Build cascaded notch filter
```

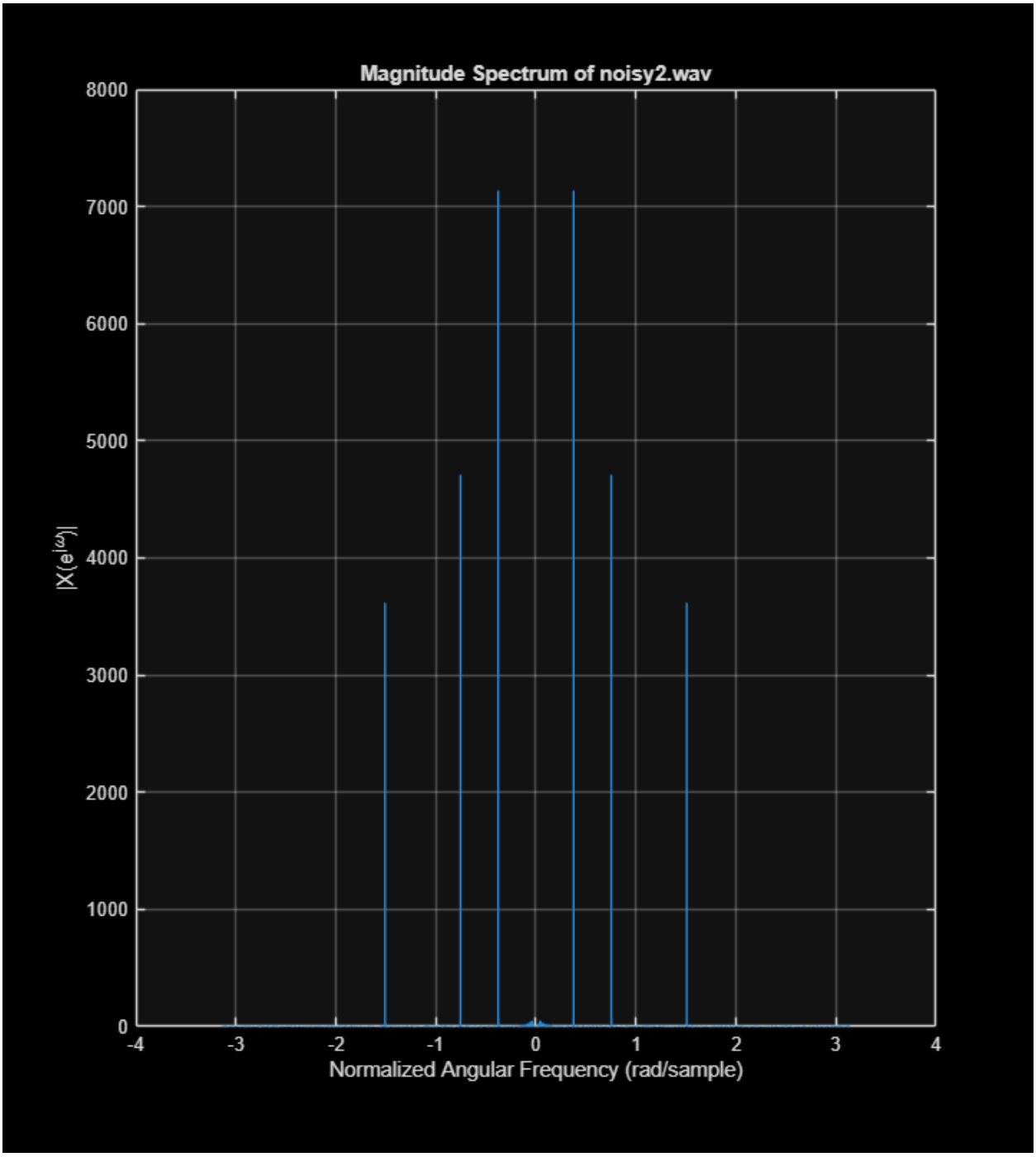
```
h_total = 1;
for k = 1:length(w_noise)
    hk = [1; -2*cos(w_noise(k)); 1];
    h_total = conv(h_total, hk);
end

% Filter and save
y2 = conv(x, h_total);
y2 = y2 / max(abs(y2));
audiowrite('denoised2.wav', y2, fs);
disp('Saved filtered audio as denoised2.wav')
```

```
Noise normalized angular frequency: 0.79262 rad/sample
Noise continuous-time cyclic frequency: 1112.643 Hz
        8820

Saved filtered audio as denoised.wav
Contaminated normalized frequencies (rad/sample):
    0.3770    0.7540    1.5080

Contaminated frequencies (Hz):
   1.0e+03 *

    0.5292    1.0584    2.1168

Saved filtered audio as denoised2.wav
```


Magnitude Spectrum of noisy.wav
15000
10000
5000
0
|X(e^jω)|
-4
-3
-2
-1
0
1
2
3
4
Normalized Angular Frequency (rad/sample)


Magnitude Response
3.5
3
2.5
2
1.5
1
0.5
0
|H(e^jω)|
-4
-2
0
2
4
Normalized Angular Frequency (rad/sample)
Phase Response
4
3
2
1
0
-1
-2
-3
∠ H(e^jω) (rad)
-4
-2
0
2
4
Normalized Angular Frequency (rad/sample)


Filtered noisy.wav Magnitude
40
35
30
25
20
15
10
5
0
|Y(e^jω)|
-4
-2
0
2
4
Normalized Angular Frequency (rad/sample)
Filtered noisy.wav Phase
4
3
2
1
0
-1
-2
-3
-4
∠ Y(e^jω) (rad)
-4
-2
0
2
4
Normalized Angular Frequency (rad/sample)


Magnitude Spectrum of noisy2.wav
8000
7000
6000
5000
4000
3000
2000
1000
0
|X(e^jω)|
-4
-3
-2
-1
0
1
2
3
4
Normalized Angular Frequency (rad/sample)

5. AI FOR FILTERING

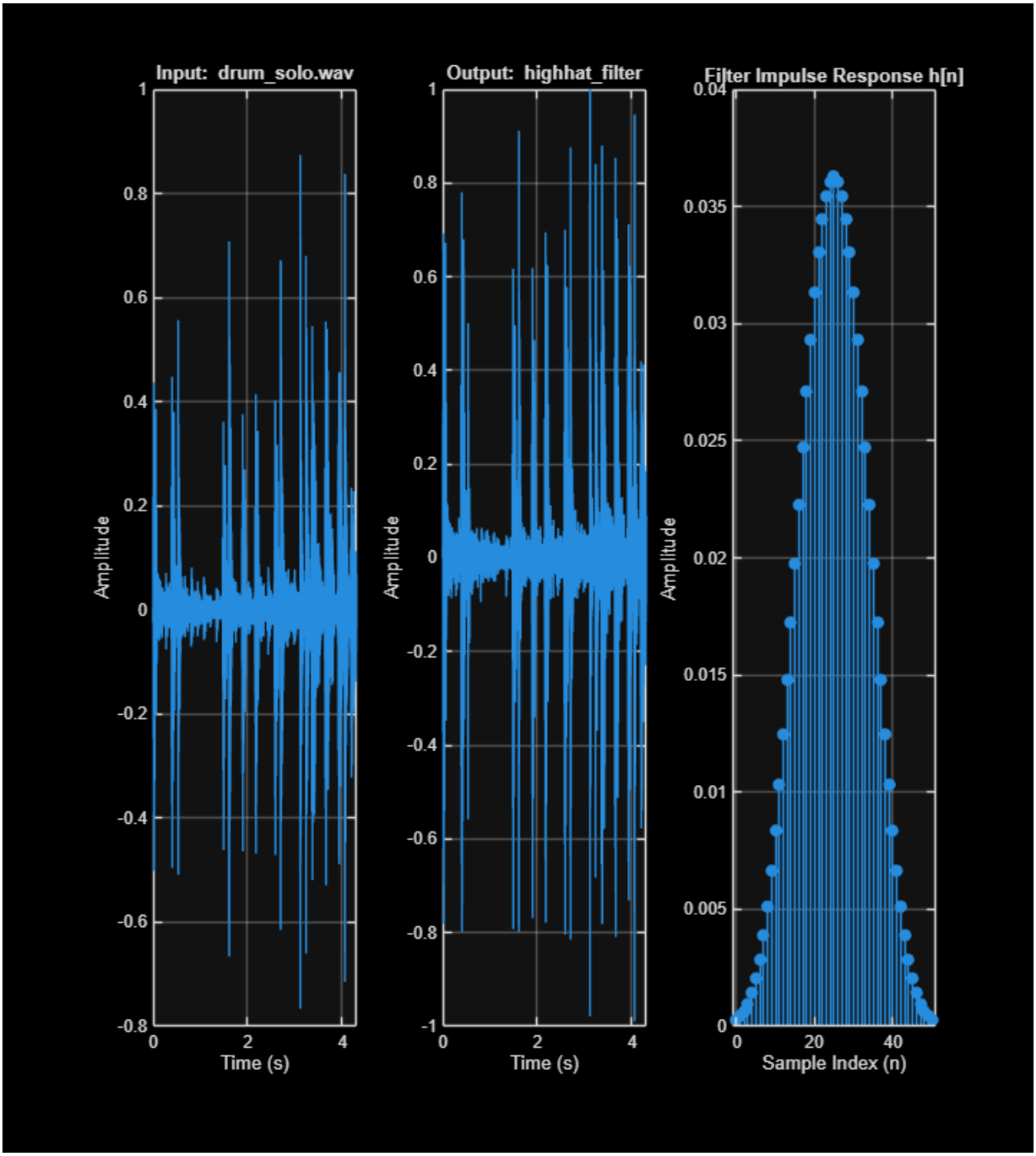
---------- 5(a) ---------- Create a function [y, h] = highhat_filter(x) that inputs a signal x with a drum solo signal and outputs a filtered y, which removes high-frequency percussive elements. It also outputs h, the impulse response for the filter. The output signal should preserve the "thump" of the kick drum while eliminating the "shimmer" of the hi-hats. Below are the specific requirements: • The filter must be a finite impulse response (FIR) filter. • The filter must be implemented as a difference equation rather than conv or filter (this is how you would implement a real-time system). Use the EEL 3135 design assistant (https://assistant.ai.it.ufl.edu/chat?assistant Id=198), like in Lab 4. You may need to collaborate with the AI to select some design details

```
% Load drum solo
[x_drum, fs_drum] = audioread('Drum_Solo.wav');
x_drum = x_drum(:,1);    % use one channel if stereo

% highhat_filter function is defined at the bottom
[y_drum, h_drum] = highhat_filter(x_drum);


% ------------------------------------------------------------
% 5(b)
% ------------------------------------------------------------
% Answer in your comments: Should this be a high-pass, low-pass, band-pass,
or band-stop filter?

% This should be a LOW-PASS filter. We want to preserve the low-frequency
% "thump" of the kick drum and remove the high-frequency
% "shimmer" of the hi-hats.




% ------------------------------------------------------------
% 5(c)
% ------------------------------------------------------------
 % Answer in your comments: Include your initial prompt and the first
question / answer


% Question #5: AI for Drum Filtering.
% (a) Create a function  [y, h] = highhat_filter(x)  that inputs a signal x
with a drum solo
```

```
% signal and outputs a filtered y, which removes high-frequency percussive
elements. It also
% outputs h, the impulse response for the filter. The output signal should
preserve the "thump"
% of the kick drum while eliminating the "shimmer" of the hi-hats. Below are
the specific
% requirements:
% • The filter must be a finite impulse response (FIR) filter.
% • The filter must be implemented as a difference equation rather than conv
or filter
% (this is how you would implement a real-time system).
% Use the EEL 3135 design assistant (https://assistant.ai.it.ufl.edu/chat?
assistant
% Id=198), like in Lab 4. You may need to collaborate with the AI to select
some design details.
%
% I need to design a low-pass FIR filter in MATLAB for a drum signal
sampled at 44100 Hz. The filter should preserve kick drum frequencies (below
~200 Hz) and remove hi-hat frequencies (above ~8000 Hz). The filter must
be implemented as a difference equation (not conv or filter). Please help
me design the filter coefficients and write a MATLAB function [y, h] =
highhat_filter(x).
% Searched for: Question #5: AI for Drum Filtering. (a) Create a f...
% First clarification question
%
% When designing an FIR low-pass filter, the filter length (order + 1)
determines how sharply the response can transition from passband to stopband
and how much ripple you can achieve. A longer filter gives a narrower
transition band and better attenuation, but it requires more computations
per sample.
%
% Question 1:
% What is the maximum number of multiplications per output sample you are
willing to allow in real-time (i.e., what is the longest filter length you
can afford)?
%
%
% 51




% ------------------------------------------------------------
% 5(d)
% ------------------------------------------------------------

% Save (as drum_ai.wav) the output of y using the the audio drum_solo.wav as
input

y_drum = y_drum / max(abs(y_drum));
audiowrite('drum_ai.wav', y_drum, fs_drum);
disp('Saved drum_ai.wav')
```

```
% ------------------------------------------------------------------
% 5(e)
% ------------------------------------------------------------------

%  Plot side-by-side the input (drum_solo.wav) (use the plot function), the
output of your filter
% (use the plot function), and the filter impulse response (use the stem
function).

t_in  = (0:length(x_drum)-1) / fs_drum;
t_out = (0:length(y_drum)-1) / fs_drum;
n_h   = 0:length(h_drum)-1;

figure
subplot(1,3,1)
plot(t_in, x_drum)
grid on
xlabel('Time (s)')
ylabel('Amplitude')
title('Input: drum\_solo.wav')

subplot(1,3,2)
plot(t_out, y_drum)
grid on
xlabel('Time (s)')
ylabel('Amplitude')
title('Output: highhat\_filter')

subplot(1,3,3)
stem(n_h, h_drum, 'filled')
grid on
xlabel('Sample Index (n)')
ylabel('Amplitude')
title('Filter Impulse Response h[n]')

snapnow;
```

Saved drum_ai.wav



ALL FUNCTIONS SUPPORTING THIS CODE

```
function H = DTFT(x,w)
%  ===> Describe function here <===
% Computes the DTFT of signal x
% evaluated at normalized angular frequencies w.
%
%  Inputs:
```

```
%    x = discrete-time signal (vector)
%    w = frequency vector (radians/sample)
%
%  Output:
%    H = DTFT of x evaluated at frequencies w


    H = zeros(length(w),1);
    for nn = 1:length(x)
        H = H + x(nn).*exp(-1j*w.'*(nn-1));
    end

end
```

```
function [y, h] = highhat_filter(x)
% LOW-PASS FIR filter to remove hi-hat shimmer and preserve kick drum.
% Filter length: 51 taps, cutoff ~800 Hz at fs = 44100 Hz
% Implemented as a difference equation (no conv or filter)

    fs     = 44100;
    fc     = 800;                          % cutoff frequency (Hz)
    N      = 51;                           % filter length (taps)
    M      = (N-1)/2;                      % half-length
    wc     = 2*pi*fc/fs;                   % normalized cutoff (rad/sample)

    % Compute sinc-based FIR coefficients with Hamming window
    n = (0:N-1) - M;

    h = zeros(1, N);
    for k = 1:N
        if n(k) == 0
            h(k) = wc/pi;
        else
            h(k) = sin(wc*n(k)) / (pi*n(k));
        end
    end

    % Manual Hamming window
    win = 0.54 - 0.46*cos(2*pi*(0:N-1)/(N-1));
    h = h .* win;

    % Implement as difference equation
    y = zeros(length(x) + N - 1, 1);
    x_padded = [zeros(N-1, 1); x(:)];
    for nn = 1:length(x)
        for k = 1:N
```

```
            y(nn) = y(nn) + h(k) * x_padded(nn + N - k);
        end
    end
    y = y(1:length(x));                 % trim to input length
end
```

2(a): predominantly LOW frequency (peak at 0 rad)
2(b): predominantly LOW frequency (peak at 0 rad)
2(c): predominantly HIGH frequency (peaks at + or - pi rad)
2(d): predominantly NEITHER (closer to low - peaks around + or - 0.8 rad
2(e): predominantly HIGH frequency (peaks around + or - 2.3 rad: I could also see this as neither, but did not know which to put.
2(f): predominantly HIGH frequency (peaks at + or - pi rad)

Published with MATLAB® R2025b